

SE-801: Software Project Lab III

Final Project Report

CodePulse

Intelligent Multi-Platform Coding Performance Analyzer

Software Design · Implementation · Testing · User Manual

Submitted by

Abdus Salam Islam Badhon

Student ID: BSSE-1401

Supervised by

Mohd. Zulfiquar Hafiz

Professor, Institute of Information Technology
University of Dhaka

Source repository: <https://github.com/Badhon1401/SPL-3>

October 2026

Letter of Transmittal

October 9, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Final Report Submission - CodePulse: Intelligent Multi-Platform Coding Performance Analyzer

Sir,

I am submitting the final report of the project “CodePulse: Intelligent Multi-Platform Coding Performance Analyzer” for SE-801: Software Project Lab III. CodePulse synchronises a learner’s submissions from Codeforces, LeetCode, AtCoder and CodeChef, computes a unified CodePulse Rating, topic-level weakness scores and hidden insights, and recommends what to practise next using both an algorithmic engine and an AI Coach built on a resilient Groq / Mistral / OpenRouter provider chain.

This report contains the requirements, system and data models, component-level design, interface design with screenshots of the running application, implementation details, AI engineering design, the test plan with test cases and outcomes, the user manual, the installation guide and the API reference. The source code is available in the public repository <https://github.com/Badhon1401/SPL-3>. I have tried to do my best and I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Abdus Salam Islam Badhon

BSSE-1401

Institute of Information Technology

University of Dhaka

Supervisor’s Signature

Mohd. Zulfiqar Hafiz

Professor, Institute of Information Technology, University of Dhaka

Acknowledgment

I am very thankful to the Institute of Information Technology, University of Dhaka, for giving me the opportunity to work on Software Project Lab III and to build CodePulse, an intelligent multi-platform coding performance analyzer.

I would like to express my sincere gratitude to my supervisor, Mohd. Zulfiquar Hafiz, Professor at the Institute of Information Technology, University of Dhaka. His continuous guidance, valuable feedback and encouragement throughout the design, maintenance and documentation of this project helped me to complete it with confidence.

Abdus Salam Islam Badhon

BSSE-1401

Institute of Information Technology, University of Dhaka

Abstract

Competitive programmers and computer-science students practise across several online judges, yet each platform shows only its own isolated statistics. Learners therefore lack a combined performance view, cross-platform weakness detection, personalised practice guidance and any visibility of behavioural trends such as streak loss or comfort-zone stagnation.

CodePulse addresses these gaps as a three-tier web application. A Next.js 14 frontend presents dashboards and charts; a Spring Boot 3.3 backend synchronises submissions from Codeforces, LeetCode, AtCoder and CodeChef, stores them in PostgreSQL (hosted on Neon), and computes a unified **CodePulse Rating** with a weighted Elo-like algorithm, combined streaks, topic-level weakness scores and automatically generated hidden insights. An algorithmic recommendation engine selects unsolved problems from a user's weakest topics, and an **AI Coach** converts free-text goals into personalised problem lists grounded in the user's measured history. The AI Coach is served by a resilient provider chain (Groq, Mistral AI and OpenRouter) that discovers usable models automatically, remembers failing models, accepts only validated JSON and falls back from model to model and provider to provider. Security is provided by BCrypt and stateless JWT authentication; data growth is bounded by a 300-submission pruning policy; and external failures are isolated so that one unavailable platform or provider never stops the others.

This report documents the complete project: requirements, system and data modeling, component-level design (design classes, persistent data sources, behavioural representations, deployment and alternatives), interface design with screenshots of the running application, implementation, AI engineering, a 17-case automated test suite, a user manual, installation guide and API reference. The source code is available at <https://github.com/Badhon1401/SPL-3>.

Keywords: competitive programming, performance analytics, recommendation engine, large language model, Groq, Mistral, OpenRouter, Spring Boot, Next.js, JWT, PostgreSQL.

Table of Contents

Letter of Transmittal	i
Acknowledgment	ii
Abstract	ii
Table of Contents	iii
List of Figures	iii
List of Tables	iii
1. Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Motivation	1
1.4 Objectives	1
1.5 Scope	2
1.6 Deliverables	2
1.7 Mapping to the SE-801 Final Requirements	2
2. Background and Related Work	3
2.1 Positioning of CodePulse	3
3. Requirements Analysis	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements	4
3.3 Stakeholders	5
3.4 Assumptions and Constraints	5
4. System Modeling	5
4.1 Use Case Modeling	6
4.2 Activity Diagrams	9
5. Data and Information Modeling	11
5.1 Database Schema	11
5.2 Entity Details	12
5.3 ER Diagram	12
5.4 Data Integrity and Retention Rules	13
6. Software Design - Component Level	13
6.1 Architectural Overview	13
6.2 Package Structure	14
6.3 Design Classes of the Problem Domain	15
6.4 Service-Layer Design Classes	15
6.5 Persistent Data Sources	17
6.6 Behavioural Representations	17
6.7 Deployment Diagram	19
6.8 Core Algorithms	19
6.9 Design Alternatives and Refactoring	21
7. Interface Design	22
7.1 Design Principles	22
7.2 Pages and Routes	22
7.3 User-Interface Objects and Actions	22
7.4 Events and State Changes	23
7.5 Screen Designs	24
8. Implementation	26
8.1 Technology Stack	26
8.2 Backend Configuration	27
8.3 Security Implementation	27
8.4 Data Synchronisation Implementation	28
8.5 Analytics Implementation	28
8.6 Recommendation Implementation	29
8.7 Frontend Implementation	29
8.8 Implementation Challenges and Resolutions	30

9. AI Engineering Design	31
9.1 Goals	31
9.2 Pipeline	31
9.3 Provider and Model Selection	31
9.4 AI Provider Layer: Design Classes	32
9.5 Failure Memory and Fallback Strategy	32
9.6 Response Validation (AiJsonUtil)	32
10. Testing	33
10.1 Testing Approach and Strategy	33
10.2 Item Pass / Fail Criteria	33
10.3 Risks and Contingencies	34
10.4 Test Cases and Results	34
10.5 Detailed Test Outcomes (Selected Cases)	35
10.6 Requirements Traceability	35
10.7 Test Summary	36
11. User Manual	36
11.1 Getting Started	36
11.2 Using the Application	37
11.3 Understanding the Numbers	38
11.4 Troubleshooting	38
12. Installation and Deployment Guide	39
12.1 Prerequisites	39
12.2 Obtain the Source	39
12.3 Backend	39
12.4 Frontend	39
12.5 Verifying the Installation	40
13. API Reference	40
13.1 Endpoint Summary	40
14. Conclusion and Deliverables	40
14.1 Summary of Achievements	40
14.2 Repository and Deliverables	41
References	41

List of Figures

Figure 1: Level-0 Use Case Diagram	6
Figure 2: Use Case Diagram - User Authentication & Profile Configuration	7
Figure 3: Use Case Diagram - Multi-Platform Data Sync & Pruning	7
Figure 4: Use Case Diagram - Analytics Computation & Hidden Insights Engine	8
Figure 5: Use Case Diagram - Algorithmic Recommendation Generation & Feedback	8
Figure 6: Use Case Diagram - AI Coach Consultation & Session Persistence	9
Figure 7: Activity Diagram - Authentication & Profile Configuration	9
Figure 8: Activity Diagram - Multi-Platform Async Synchronisation & Pruning	10
Figure 9: Activity Diagram - Analytics Engine & CodePulse Rating Pipeline	10
Figure 10: Activity Diagram - Algorithmic Recommendation & Feedback Pipeline	10
Figure 11: Activity Diagram - AI Coach Prompt-to-Recommendation Pipeline	11
Figure 12: Entity–Relationship Diagram	13
Figure 13: Component Diagram	14
Figure 14: Domain (entity) class diagram	15
Figure 15: Service-layer class diagram	16
Figure 16: Sequence diagram - GET /api/analytics/me	18
Figure 17: Sequence diagram - POST /api/ai/recommend	18
Figure 18: Deployment diagram	19
Figure 19: State diagram - Dashboard	23
Figure 20: State diagram - AI Coach page	23
Figure 21: Login and two-step registration screens	24
Figure 22: Registration step 1: username and full name	24
Figure 23: Registration step 2: email and password	24
Figure 24: Dashboard screen	24
Figure 25: Analytics screen	25
Figure 26: Analytics: weakness radar, verdict distribution and topic coverage	25
Figure 27: Recommendations screen	25
Figure 28: AI Coach screen	26
Figure 29: AI Coach: recommended problems with focus areas and reasons	26
Figure 30: Profile screen	26
Figure 31: Frontend project structure (Next.js app router)	30
Figure 32: AI pipeline flow	31
Figure 33: Class diagram of the AI provider layer	32
Figure 34: Login and registration screens	36
Figure 35: Registration (step 2: credentials)	36
Figure 36: AI Coach	38

List of Tables

Table 1: Project scope	2
Table 2: SE-801 requirements and where they are addressed	2
Table 3: Capability comparison	3
Table 4: Authentication and user management requirements	3
Table 5: Data synchronisation requirements	3
Table 6: Analytics requirements	4
Table 7: Recommendation requirements	4
Table 8: AI Coach requirements	4
Table 9: Non-functional requirements	4
Table 10: Stakeholders	5
Table 11: Primary use cases	6
Table 12: Use case description UC-1	6
Table 13: Use case description UC-2	7
Table 14: Use case description UC-3	7

Table 15: Use case description UC-4	8
Table 16: Use case description UC-5	8
Table 17: Database tables	11
Table 18: Selected entity attributes	12
Table 19: Layers and responsibilities	14
Table 20: Domain classes and responsibilities	15
Table 21: Service-layer classes	16
Table 22: Repository queries	17
Table 23: Recommendation lifecycle	18
Table 24: Deployment nodes	19
Table 25: Per-platform rating formulas	19
Table 26: Hidden insights rule set	20
Table 27: Design alternatives considered	21
Table 28: Pages, routes and UI objects	22
Table 29: Interface objects and operations	22
Table 30: UI events and state transitions	23
Table 31: Technology stack	26
Table 32: Key configuration properties	27
Table 33: Security controls	28
Table 34: Analytics response structure	28
Table 35: Implementation problems and fixes	30
Table 36: AI pipeline stages	31
Table 37: Model selection	31
Table 38: Cooldowns applied by ModelHealthRegistry	32
Table 39: Testing levels	33
Table 40: Test environment	33
Table 41: Pass / fail criteria	33
Table 42: Risks and contingencies	34
Table 43: Test cases	34
Table 44: Execution evidence from the automated run	35
Table 45: Requirements-to-test traceability	35
Table 46: Test summary	36
Table 47: Glossary of displayed metrics	38
Table 48: Troubleshooting guide	38
Table 49: Prerequisites	39
Table 50: Smoke test	40
Table 51: REST endpoints	40
Table 52: Final deliverables	41

1. Introduction

1.1 Project Overview

CodePulse is a full-stack web application that acts as an analytical intelligence layer above the major competitive-programming platforms - Codeforces, LeetCode, AtCoder and CodeChef. Instead of forcing a learner to read four separate statistics pages, CodePulse synchronises their submission history, normalises it into a single relational model, and derives insights that none of the individual platforms provide: a unified **CodePulse Rating**, cross-platform topic-level weakness scores, a combined streak, automatically generated "hidden insights", algorithmic problem recommendations, and an **AI Coach** that turns a free-text goal into a personalised practice plan.

The system is built as a three-tier application. The presentation tier is a Next.js 14 single-page style interface; the application tier is a Spring Boot 3.3 REST backend that contains the synchronisation, analytics, recommendation and AI-integration engines; and the data tier is a PostgreSQL database. Stateless JWT authentication secures every layer boundary.

This document is the **final report** for the SE-801 project course. It consolidates the earlier deliverables (proposal and midterm SRS) and extends them with the component-level design, interface design, implementation details, testing documentation, user manual and installation guide required for the final defence.

1.2 Problem Statement

Competitive programmers and software engineering students practise on several platforms at once. Each platform operates in isolation and presents only its own statistics. As a result learners face the following concrete problems:

- There is **no single place** to see combined solved-problem counts, acceptance rates or activity trends across all platforms.
- There is **no automated weakness detection** that spans platforms - for example, repeatedly failing Dynamic Programming problems on both Codeforces and LeetCode.
- Choosing the next problem is **ad hoc**. Users cannot state a goal in natural language and receive a personalised, history-aware practice list.
- There is **no unified rating** that reflects competence across all connected platforms.
- Behavioural patterns - declining activity, comfort-zone stagnation, streak loss - are **invisible** without manual record keeping.

1.3 Motivation

Mainstream platforms excel at hosting problems and contests but give little structured feedback on *how* a user is improving. Learning becomes inefficient and repetitive when practice is not guided by data. CodePulse targets three groups: competitive programmers who want to level up, computer-science students preparing for assessments, and job candidates preparing for technical interviews. The core purpose is to convert scattered statistics into intelligent, actionable guidance for structured skill development.

1.4 Objectives

- Build a full-stack web application that aggregates data from Codeforces, LeetCode, AtCoder and CodeChef into one dashboard.
- Compute a unified **CodePulse Rating** with a weighted Elo-like algorithm.
- Implement automated topic-level weakness detection and a curated problem recommendation engine with a feedback loop.

- Integrate Large Language Models (Groq, Mistral AI, OpenRouter) through a resilient provider chain with automatic model discovery, failure memory and strict JSON validation, to accept free-text coaching prompts and return recommendations grounded in the user's real performance history.
- Generate **hidden analytical insights** - patterns the user would not otherwise notice.
- Provide scheduled (6-hourly) and on-demand synchronisation with a configurable ceiling of 300 stored submissions per user.
- Secure the system with BCrypt password hashing, stateless JWT authentication and role-based authorisation.

1.5 Scope

Table 1: Project scope

In scope	Out of scope
Registration, login, profile with four platform handles Synchronisation with Codeforces REST, LeetCode GraphQL, AtCoder (kenkoooo) and CodeChef community APIs Combined analytics, rating, streaks, heatmap, per-platform breakdown, hidden insights Algorithmic recommendations AI Coach with persisted sessions Automatic 6-hour sync, submission pruning	Contest scheduling and reminders Real-time editorial fetching Social / collaborative features Native mobile application Executing or judging user code

1.6 Deliverables

- Full-stack application: Spring Boot 3.3 backend and Next.js 14 frontend.
- PostgreSQL schema with 7 core tables (plus the `problem_topics` join table).
- REST API with 18 documented endpoints (including an AI self-test endpoint).
- This final report (design, implementation, testing, user manual, repository URL).
- Source repository: <https://github.com/Badhon1401/SPL-3>
- Installer: not applicable - the system is run from source / packaged JAR (see Chapter 12).

1.7 Mapping to the SE-801 Final Requirements

The SE-801 final-presentation brief lists the components the final report must cover. Table 2 maps every item of the brief to the chapter of this report that addresses it.

Table 2: SE-801 requirements and where they are addressed

SE-801 requirement	Addressed in	Status
Component-level design: design classes of the problem domain	Sections 6.3, 6.4 and 9.4 (domain, service and AI-provider class diagrams)	Complete
Persistent data sources and their classes	Chapter 5, Section 6.5 (database, repositories, lazy-loading strategy)	Complete
Behavioural representations of classes / components	Section 4.2 (activity diagrams), Section 6.6 (sequence, lifecycle, sync behaviour)	Complete
Deployment diagram with implementation detail	Section 6.7 (Figure 18, Table 24)	Complete
Refactoring and design alternatives	Sections 6.9 and 8.8	Complete
Interface design: UI objects and actions	Sections 7.2 and 7.3	Complete
Interface design: events that change UI state	Section 7.4 (Table 30, state diagrams)	Complete
Interface design: each state as the end user sees it	Section 7.5 and Chapter 11 (screenshots of the running application)	Complete

SE-801 requirement	Addressed in	Status
Testing approach, pass/fail criteria, risks and contingencies	Sections 10.1 to 10.3	Complete
Test cases with detailed outcomes	Sections 10.4 to 10.7 (17 test cases)	Complete
User manual	Chapter 11	Complete
URL of public version-controlled repository	Sections 1.6 and 14.2: https://github.com/Badhon1401/SPL-3	Complete
Installer (if applicable)	Not applicable; installation from source is described in Chapter 12	N/A
Full demonstration (acceptance test) and code review	Prepared for the defence; the demonstration follows the user manual (Chapter 11)	Ready

2. Background and Related Work

2.1 Positioning of CodePulse

Table 3: Capability comparison

Capability	Single-platform profile	CodePulse
Cross-platform aggregation	No	Yes - four platforms, one dashboard
Unified rating	No	Yes - weighted Elo-like CodePulse Rating
Topic weakness scoring	Limited / manual	Automatic failure-rate score per topic
Hidden insights (comfort zone, streak drop, imbalance)	No	Yes - 6–10 categorised findings
Personalised recommendations	Generic problem lists	Algorithmic + AI Coach grounded in history
Natural-language practice goals	No	Yes - LLM coach (Groq / Mistral / OpenRouter)

3. Requirements Analysis

3.1 Functional Requirements

3.1.1 Authentication and User Management

Table 4: Authentication and user management requirements

ID	Requirement	Use case
FR-1	Users can register with username, email and password; passwords are stored BCrypt-hashed.	UC-1
FR-2	Users can log in and receive a JWT access token with a 24-hour expiry.	UC-1
FR-3	Users can store and update Codeforces, LeetCode, AtCoder and CodeChef handles in their profile.	UC-1
FR-4	Platform handles are always fetched fresh from the database on profile and dashboard load.	UC-1

3.1.2 Data Synchronisation

Table 5: Data synchronisation requirements

ID	Requirement	Use case
FR-5	Fetch up to 200 recent submissions per platform per sync from Codeforces (<code>/user.status</code>), LeetCode (GraphQL), AtCoder (kenkoooo) and CodeChef (community API).	UC-2
FR-6	Deduplicate submissions by <code>platformSubmissionId</code> before storage.	UC-2
FR-7	If a user's stored submissions exceed 300, prune the oldest so only the most recent 300 remain.	UC-2

ID	Requirement	Use case
FR-8	Synchronise automatically every 6 hours via a scheduled job and allow manual triggering.	UC-2

3.1.3 Analytics

Table 6: Analytics requirements

ID	Requirement	Use case
FR-9	Compute a combined CodePulse Rating from all connected platforms using a weighted-average algorithm.	UC-3
FR-10	Compute current streak, longest streak and total active days across all platforms combined.	UC-3
FR-11	Compute topic-level weakness scores (failure rate per topic) from all submissions.	UC-3
FR-12	Generate 6–10 hidden insights automatically (weakness, comfort zone, streak pattern, activity trend, platform imbalance ...).	UC-3
FR-13	Return per-platform statistics (solved count, acceptance rate, verdict distribution, topic coverage) alongside combined statistics.	UC-3

3.1.4 Recommendations

Table 7: Recommendation requirements

ID	Requirement	Use case
FR-14	Select up to 10 unsolved problems from the user's weakest topics within the estimated difficulty range.	UC-4
FR-15	Generating new recommendations replaces all previous ones for that user.	UC-4
FR-16	Users can mark a recommendation as solved (feedback loop) or dismiss it.	UC-4

3.1.5 AI Coach

Table 8: AI Coach requirements

ID	Requirement	Use case
FR-17	Users can submit a free-text prompt describing a practice goal.	UC-5
FR-18	Build a context payload (rating, weakness scores, per-platform stats, recently solved problems, recent WA/TLE topics) and send it with the prompt to the configured LLM providers.	UC-5
FR-19	The model returns structured JSON: problem recommendations, a coach insight paragraph and focus areas.	UC-5
FR-20	Persist AI sessions; the latest session is shown on page load without regeneration.	UC-5
FR-21	Regeneration deactivates the previous session and creates a new one.	UC-5
FR-22	Users can mark AI-recommended items as solved or dismiss them.	UC-5
FR-23	Support Groq, Mistral AI and OpenRouter; a provider is used only if its API key is configured, and providers are ordered automatically (last working first, cooling-down last).	UC-5
FR-24	Discover the chat models available to each key at run time, try them best-first and put failed models on a cooldown (ModelHealthRegistry); no model names need configuring.	UC-5
FR-25	Accept an LLM answer only if it holds a JSON object with a non-empty recommendations array; strip <think> blocks and code fences; treat anything else as a failure and try the next model or provider.	UC-5
FR-26	Keep a recommendation URL only if it points to Codeforces, LeetCode, AtCoder or CodeChef; otherwise generate a search link.	UC-5
FR-27	Provide authenticated self-test endpoints (/api/ai/health/{provider}) for each provider.	UC-5

3.2 Non-Functional Requirements

Table 9: Non-functional requirements

ID	Attribute	Requirement	Measure
NFR-1	Performance	Analytics responses for a user with up to 300 submissions are returned quickly.	< 2 s
NFR-2	Security	All endpoints except <code>/api/auth/**</code> require a valid JWT; passwords never stored in plaintext.	401 without / with invalid token
NFR-3	Reliability	A failure on one platform must not crash the overall sync job.	Error caught and logged; other platforms continue
NFR-4	Scalability	Database growth per user is bounded; syncs are asynchronous.	≤ 300 submission rows per user
NFR-5	Usability	Loading skeletons, toast notifications and empty states with action prompts.	Manual acceptance test
NFR-6	Maintainability	Each platform sync is an independent service class.	New platform = 1 service + 1 handle field
NFR-7	AI correctness	LLM prompt demands JSON only; responses are validated by <code>AiJsonUtil</code> and parsed through <code>JsonNode</code> .	Tolerates additive schema changes
NFR-8	Portability	Runs on any OS with Java 17, PostgreSQL 15 and Node.js 18.	Verified on development machine
NFR-9	AI availability	A slow or failing model must not hang the request: per-model timeouts (10 s connect, 45–60 s read), at most 10 OpenRouter attempts within 90 s, no database transaction held during the LLM call.	Friendly error or result; no stack trace

3.3 Stakeholders

Table 10: Stakeholders

Stakeholder	Role	Interest in CodePulse
Competitive programmer	Primary user	Improve problem-solving skills efficiently
CS student	Primary user	Prepare for coding interviews, contests and assessments
Supervisor / evaluator	Academic oversight	Assess technical depth, design quality and engineering rigour
Developer / maintainer	Builder	Clean architecture and an extensible platform
External platform providers	Data source	Fair-use, rate-limited API access

3.4 Assumptions and Constraints

- Platform handles are supplied by the user; CodePulse reads only **public** submission data and never needs platform passwords.
- Public APIs may rate-limit or change; the design therefore isolates each platform behind its own service and tolerates partial failure.
- At most 200 submissions are fetched per platform per sync and 300 are retained per user, so analytics reflect **recent** performance rather than complete history.
- The AI Coach requires an internet connection and an API key for at least one provider (Groq, Mistral AI or OpenRouter); providers without a key are skipped automatically.
- The project was developed by a single student within one academic semester (17 weeks).

4. System Modeling

System modeling provides the behavioural view of CodePulse. This chapter presents the Level-0 use case diagram, detailed descriptions of the five principal use cases, and activity diagrams for each core execution pipeline across the Spring Boot backend, the PostgreSQL persistence layer and the external integrations.

4.1 Use Case Modeling

4.1.1 Level-0 System Use Case Diagram

The diagram shows the interaction between the primary human actor (Authenticated User), the automated System Scheduler, the system boundary, and the external services (Competitive Programming APIs and the AI provider chain (Groq, Mistral AI, OpenRouter)).

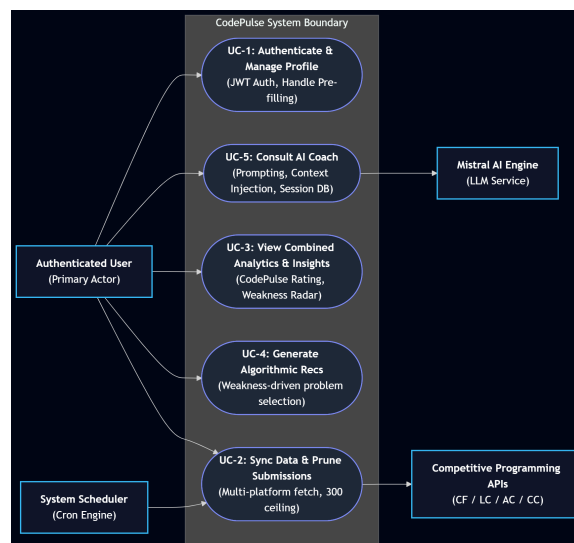


Figure 1: Level-0 Use Case Diagram

Table 11: Primary use cases

Use case	Summary
Register / Login	Create an account and authenticate; a 24-hour JWT is issued.
Manage Profile	Add or update platform handles and view connection status.
Sync Data	Fetch, deduplicate and store submissions from all connected platforms (manual or scheduled).
View Dashboard	See CodePulse Rating, streak, platform connections and recent submissions.
View Analytics	Explore charts, per-platform breakdown and hidden insights.
Get Curated Recommendations	Browse algorithmically selected problems; mark solved or dismiss.
Ask AI Coach	Enter a free-text goal and receive personalised AI-generated problems.
Administer Users	Administrators list and deactivate users (ROLE_ADMIN).

4.1.2 UC-1: Authentication and Profile Handle Configuration

Table 12: Use case description UC-1

Field	UC-1 Authentication & Profile Handle Configuration
Actor	Authenticated User
Trigger	User opens the Register / Login page or updates profile settings (<code>/profile</code>).
Preconditions	Backend is operational; PostgreSQL connection is active.

Field	UC-1 Authentication & Profile Handle Configuration
Main flow	1. User registers with email, username and password, or logs in. 2. Backend verifies credentials via BCrypt and issues a 24-hour signed JWT. 3. User enters or updates handles (Codeforces, LeetCode, AtCoder, CodeChef) in Profile. 4. Backend persists the handles in the <code>users</code> table. 5. System triggers an initial asynchronous sync for new handles.
Post-conditions	JWT stored in client state; handles saved and ready for cross-platform fetching.
Exceptions	Invalid credentials → 401 Unauthorized. Duplicate email / username → 400 / 409 with message.

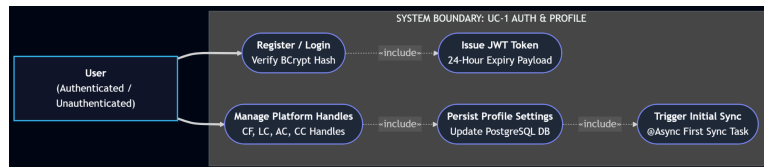


Figure 2: Use Case Diagram - User Authentication & Profile Configuration

4.1.3 UC-2: Multi-Platform Data Sync and Pruning

Table 13: Use case description UC-2

Field	UC-2 Multi-Platform Data Sync & Pruning
Actor	Authenticated User / Background Scheduler (<code>ScheduledSyncService</code>)
Trigger	User clicks "Sync All Platforms" (<code>POST /api/analytics/sync</code>) or the 6-hour cron fires.
Preconditions	User profile contains at least one connected platform handle.
Main flow	1. Parallel <code>@Async</code> requests are sent to the connected platform APIs. 2. Up to 200 recent submissions are fetched per platform. 3. Incoming submissions are deduplicated by <code>platformSubmissionId</code>. 4. New <code>Problem</code> and <code>Topic</code> records are created if absent. 5. <code>SubmissionPruningService</code> deletes the oldest submissions if the user has more than 300. 6. <code>lastSyncedAt</code> is updated on the user record.
Post-conditions	Database holds ≤ 300 recent submissions for the user; analytics are recalculated on demand.
Exceptions	Platform timeout / rate limit → error logged for that platform; remaining platforms continue.

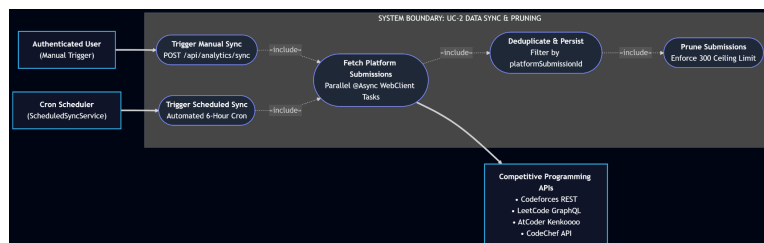


Figure 3: Use Case Diagram - Multi-Platform Data Sync & Pruning

4.1.4 UC-3: Analytics Computation and Hidden Insights

Table 14: Use case description UC-3

Field	UC-3 Analytics Computation & Hidden Insights Engine
Actor	Authenticated User
Trigger	User opens the Dashboard or Analytics page (<code>GET /api/analytics/me</code>).
Preconditions	Valid JWT; submissions have been synchronised.

Field	UC-3 Analytics Computation & Hidden Insights Engine
Main flow	1. <code>AnalyticsServiceImpl</code> loads the user's submissions. 2. <code>CombinedRatingCalculator</code> computes per-platform ratings and the weighted CodePulse Rating. 3. Current streak, longest streak and active days are computed. 4. Topic failure rates produce strength / weakness distributions. 5. The Hidden Insights Engine derives 6–10 findings. 6. A <code>PerformanceAnalyticsResponse</code> DTO is returned to the frontend.
Post-conditions	Charts, rating, streak statistics and insights are rendered.
Exceptions	No submission history → zeroed DTO with empty-state flags.

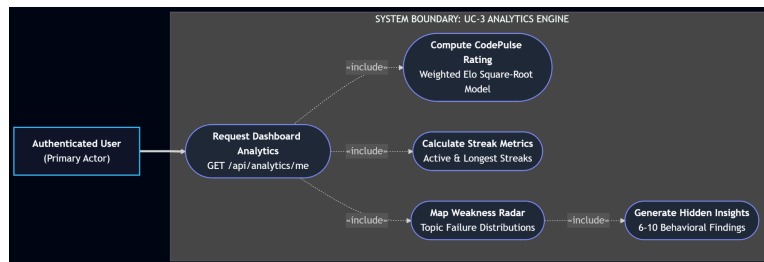


Figure 4: Use Case Diagram - Analytics Computation & Hidden Insights Engine

4.1.5 UC-4: Algorithmic Recommendation Generation and Feedback

Table 15: Use case description UC-4

Field	UC-4 Algorithmic Recommendation Generation & Feedback
Actor	Authenticated User
Trigger	User requests curated recommendations (<code>POST /api/recommendations/generate</code>).
Preconditions	Weakness scores can be computed from synchronised history.
Main flow	1. <code>RecommendationServiceImpl</code> identifies the top weakness topics. 2. The engine derives the user's difficulty bracket from the CodePulse Rating. 3. Unsolved problems in those topics within the bracket are queried. 4. Up to 10 curated problems are selected across all platforms. 5. Previous recommendations are deleted and the new ones persisted. 6. User marks items solved (<code>PATCH .../solved</code>) or dismissed (<code>PATCH .../dismiss</code>).
Post-conditions	Recommendation list updated; solved / dismissed flags persisted.
Exceptions	Insufficient problem catalogue → fall back to difficulty-matched problems.

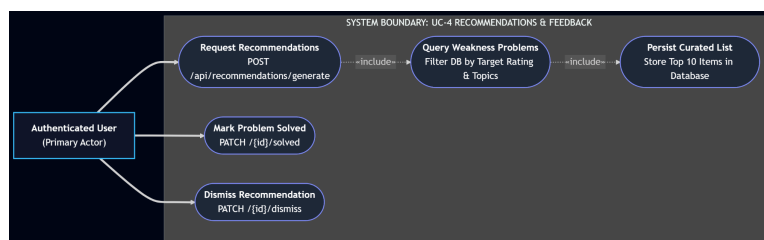


Figure 5: Use Case Diagram - Algorithmic Recommendation Generation & Feedback

4.1.6 UC-5: AI Coach Consultation and Session Persistence

Table 16: Use case description UC-5

Field	UC-5 AI Coach Consultation & Session Persistence
Actor	Authenticated User
Trigger	User submits a free-text practice prompt (<code>POST /api/ai/recommend</code>).
Preconditions	At least one AI provider key configured; user logged in.

Field	UC-5 AI Coach Consultation & Session Persistence
Main flow	- Context assembly: rating, per-platform stats, top 8 weak topics, top 5 strong topics, recent WA/TLE topics (short read-only transaction). - Prompt assembler merges the user prompt, context and a strict JSON schema. - Providers are ordered automatically (last working first, cooling-down last); for each, the usable models are discovered and tried best-first. - A response is accepted only if AiJsonUtil finds a JSON object with a non-empty recommendations array; otherwise the next model or provider is tried. - In a short write transaction the previous session is set inactive. - The new session and items are saved; URLs are sanitised. - Recommendations, coach insight, focus areas and the model used are returned.
Post-conditions	Session history stored; latest session auto-loads on page render.
Exceptions	All providers fail, invalid JSON or a persistence error → exception caught and logged; buildError() returns an empty list with a friendly message; the previous session is retained.

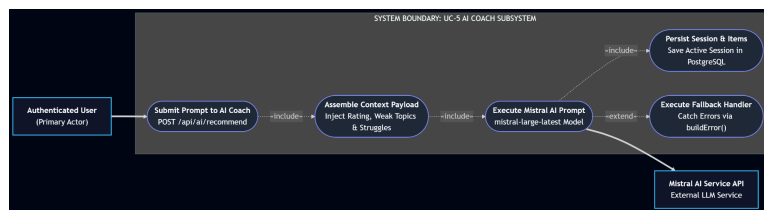


Figure 6: Use Case Diagram - AI Coach Consultation & Session Persistence

4.2 Activity Diagrams

Activity diagrams model the operational flows, branching decisions and state transitions of the five core pipelines. Each diagram is shown beside a step-by-step explanation.

4.2.1 AD-1: User Authentication and Profile Configuration

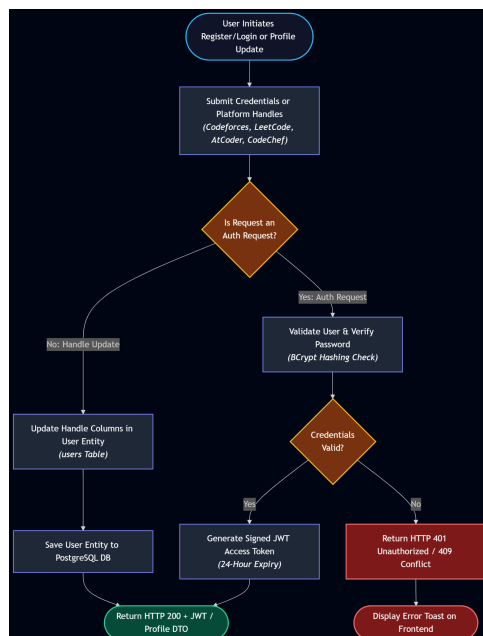


Figure 7: Activity Diagram - Authentication & Profile Configuration

This flow captures registration, BCrypt password hashing, JWT issuance and persistence of platform handles.

1. The user submits credentials or profile data.
2. A decision node distinguishes **login**, **registration** and **profile update** requests.
3. For login the password is verified against the stored BCrypt hash; failure returns **401**.
4. For registration the system checks email / username uniqueness; a duplicate returns an error, otherwise the password is hashed and the user saved.
5. A signed 24-hour JWT is generated and returned.
6. Profile updates persist the four handle columns in **users** and trigger an asynchronous initial sync.

4.2.2 AD-2: Multi-Platform Async Synchronisation and Pruning

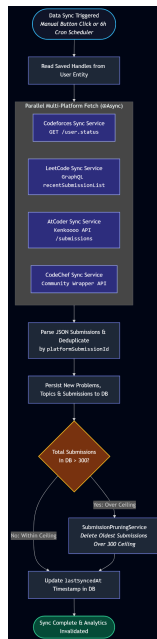


Figure 8: Activity Diagram - Multi-Platform Async Synchronisation & Pruning

This flow captures asynchronous multi-platform fetching, deduplication, storage and the 300-submission ceiling.

1. Sync is initiated by a manual click or by the 6-hour cron job.
2. Saved handles are read from the `User` entity.
3. Four `@Async` services run in **parallel**: Codeforces `/user.status`, LeetCode `recentSubmissionList`, AtCoder `kenkoooo/submissions`, CodeChef `community wrapper API`.
4. Responses are parsed and deduplicated by `platformSubmissionId`.
5. New problems, topics and submissions are persisted.
6. If total submissions > 300 , `SubmissionPruningService` deletes the oldest surplus rows.
7. `lastSyncedAt` is updated and cached analytics are invalidated.

A failure in one lane is logged and does not abort the other lanes (NFR-3).

4.2.3 AD-3: Analytics Engine and CodePulse Rating

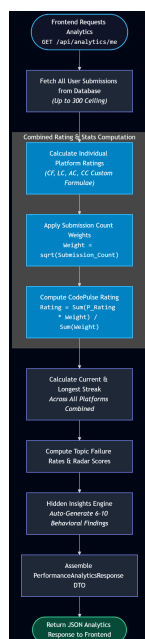


Figure 9: Activity Diagram - Analytics Engine & CodePulse Rating Pipeline

This flow details how an analytics request aggregates cross-platform data.

1. The frontend calls `GET /api/analytics/me`.
2. All submissions (≤ 300) are fetched with problems and topics in a single `JOIN FETCH` query.
3. Individual platform ratings are computed with the platform-specific formulas.
4. Weights $\sqrt{(\text{submission count})}$ are applied and the CodePulse Rating is the weighted average, clipped to $[800, 3800]$.
5. Current and longest streaks are calculated across all platforms.
6. Topic failure rates and radar scores are computed.
7. The Hidden Insights Engine derives 6–10 findings.
8. The assembled `PerformanceAnalyticsResponse` is returned.

4.2.4 AD-4: Algorithmic Recommendation and Feedback

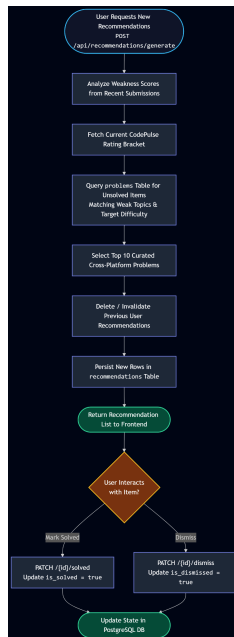


Figure 10: Activity Diagram - Algorithmic Recommendation & Feedback Pipeline

This flow illustrates weak-topic detection, rating-bracket querying, invalidation of old recommendations and user feedback.

1. The user requests new recommendations.
2. Weakness scores are derived from recent submissions.
3. The current rating bracket is determined.
4. The `problems` table is queried for **unsolved** items in weak topics and the target difficulty range.
5. Up to 10 problems are selected across platforms.
6. Previous recommendations are deleted; new rows are persisted.
7. When the user interacts, **Mark Solved** sets `is_solved = true` and **Dismiss** sets `is_dismissed = true`.

4.2.5 AD-5: AI Coach Prompt-to-Recommendation

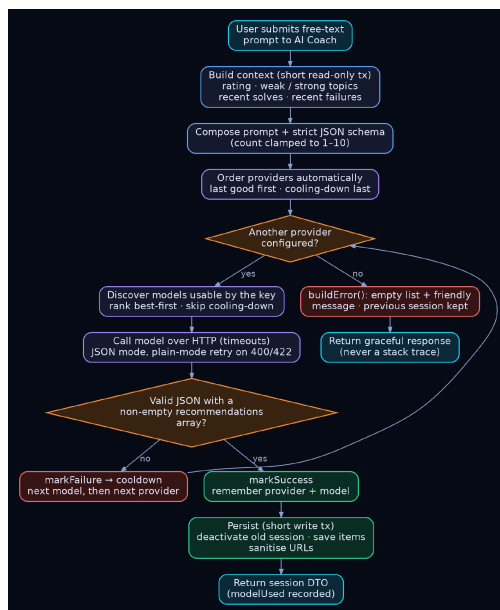


Figure 11: Activity Diagram - AI Coach Prompt-to-Recommendation Pipeline

This flow captures the full vertical pipeline from the user's prompt to a persisted session.

1. Stage 1 - context assembly (rating, platform stats, weak / strong topics, struggles).
2. Stage 2 - prompt assembly with a strict JSON schema.
3. Stage 3 - provider chain: Groq, Mistral AI and OpenRouter are ordered automatically; models are discovered per key and tried best-first with JSON mode (plain-mode retry on HTTP 400/422).
4. Stage 4 - validation: AiJsonUtil requires a JSON object with a non-empty recommendations array. Invalid output marks the model as failed (cooldown) and moves to the next model or provider; when all fail, buildError() returns a graceful response and the previous session is preserved.
5. Stage 5 - persistence: in a short write transaction the previous session is deactivated, the new session and items (with sanitised URLs) are saved and the result is returned to the UI.

5. Data and Information Modeling

5.1 Database Schema

CodePulse uses PostgreSQL 15 with seven core tables plus one join table. All tables use auto-generated `BIGINT` primary keys with foreign-key constraints. The schema is generated and evolved by Hibernate

(`ddl-auto=update`). Open-session-in-view is disabled, so every lazy association is loaded explicitly (see Section 6.5).

Table 17: Database tables

Table	Key columns	Purpose
<code>users</code>	id, username, email, password_hash, full_name, avatar_url, codeforces_handle, leetcode_handle, atcoder_handle, codechef_handle, role, active, last_synced_at	Authentication and the four platform handles
<code>problems</code>	id, platform_id, platform, title, difficulty_rating, difficulty_label, problem_url	Cross-platform problem catalogue
<code>topics</code>	id, name, slug	Algorithm topic tags (20 seeded on first boot)
<code>problem_topics</code>	problem_id (FK), topic_id (FK)	Many-to-many join between problems and topics
<code>submissions</code>	id, user_id (FK), problem_id (FK), platform_submission_id, verdict, language, submitted_at, synced_at	All synchronised submissions (pruned to 300 per user)
<code>recommendations</code>	id, user_id (FK), problem_id (FK), reason, score, is_solved, is_dismissed	Algorithmic curated recommendations
<code>ai_recommendation_sessions</code>	id, user_id (FK), prompt, coach_insight, focus_areas_json, model_used, active, created_at	One AI Coach consultation; one active per user
<code>ai_recommendation_items</code>	id, session_id (FK), title, platform, url, difficulty, estimated_rating, topics_json, reason, time_estimate, is_solved, is_dismissed	Individual problems within an AI session

5.2 Entity Details

Table 18: Selected entity attributes

Entity	Attribute	Type	Constraint / note
User	id	BIGINT	PK, generated
	username, email	VARCHAR	UNIQUE, NOT NULL
	password_hash	VARCHAR	BCrypt hash, NOT NULL
	*_handle (×4)	VARCHAR	Nullable; one per platform
	role	ENUM	ROLE_USER (default) / ROLE_ADMIN
	last_synced_at	TIMESTAMP	Updated after each successful sync
Problem	platform + platform_id	VARCHAR	Logical natural key (e.g. CF + 1850A)
	difficulty_rating	INT	Numeric where the platform provides one
	difficulty_label	VARCHAR	Beginner / Easy / Medium / Hard / Expert
Submission	platform_submission_id	VARCHAR	Used for deduplication
	verdict	VARCHAR	ACCEPTED, WRONG_ANSWER, TIME_LIMIT_EXCEEDED, ...
	submitted_at	TIMESTAMP	Basis for streaks, heatmap and pruning order
Recommendation	score	DOUBLE	Priority score shown as a progress bar
AI session	active	BOOLEAN	Exactly one active session per user
AI item	topics_json	TEXT	JSON array serialised as text

5.3 ER Diagram

Key cardinalities of the model:

- **User 1 - N Submission; Submission N - 1 Problem.**

- **Problem N - M Topic** realised through `problem_topics`.
- **User 1 - N Recommendation**, each referencing exactly one Problem.
- **User 1 - N AiRecommendationSession**, and **AiRecommendationSession 1 - N AiRecommendationItem**.

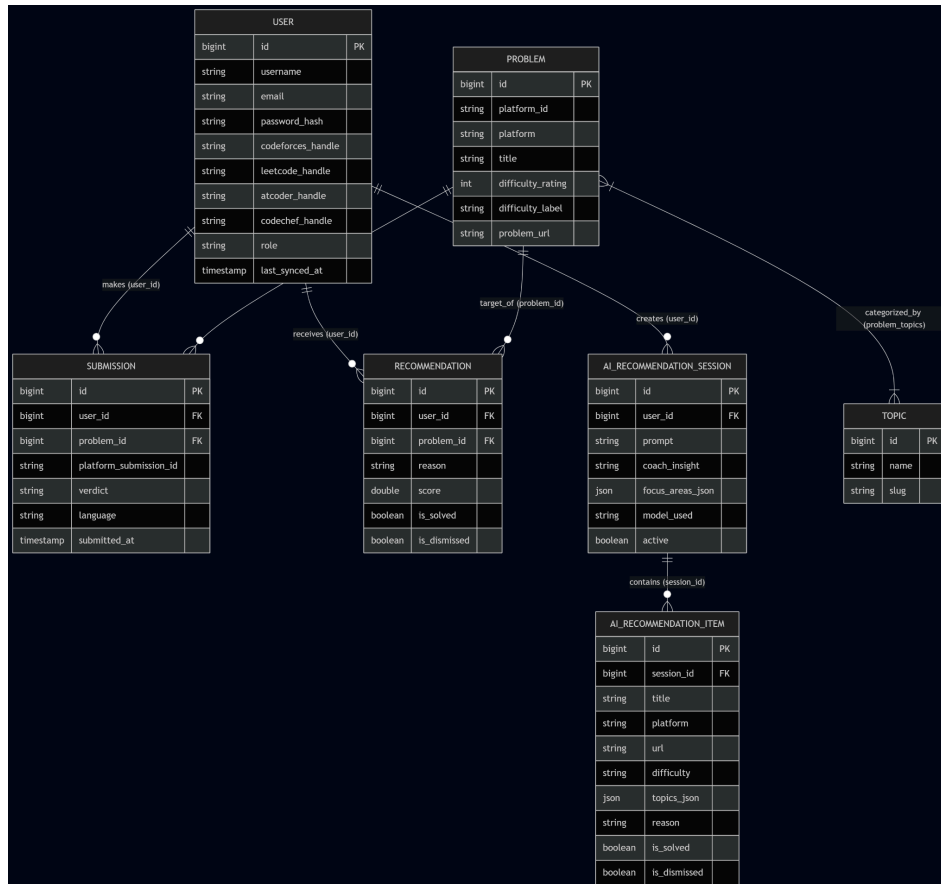


Figure 12: Entity–Relationship Diagram

5.4 Data Integrity and Retention Rules

- **Uniqueness**: a submission is identified by `(platform, platform_submission_id)`; repeated syncs never create duplicates (FR-6).
- **Bounded growth**: a user retains at most 300 submissions; the oldest by `submitted_at` are bulk-deleted after each sync (FR-7).
- **Replace-on-generate**: generating curated recommendations deletes the previous set for that user (FR-15).
- **Single active AI session**: creating a session first deactivates all earlier sessions for the user (FR-21).
- **Soft deactivation**: administrators deactivate users rather than deleting rows, preserving referential integrity.

6. Software Design - Component Level

This chapter presents the component-level design of CodePulse: the architecture, the design classes of the problem domain, the persistent data sources, behavioural representations of key components, the deployment view, the core algorithms, and a record of the alternatives that were considered and the refactorings applied.

6.1 Architectural Overview

CodePulse follows a clean **3-tier architecture** with an additional AI-integration layer and an asynchronous synchronisation layer. Separation of concerns keeps presentation, business logic and persistence independently testable and replaceable.

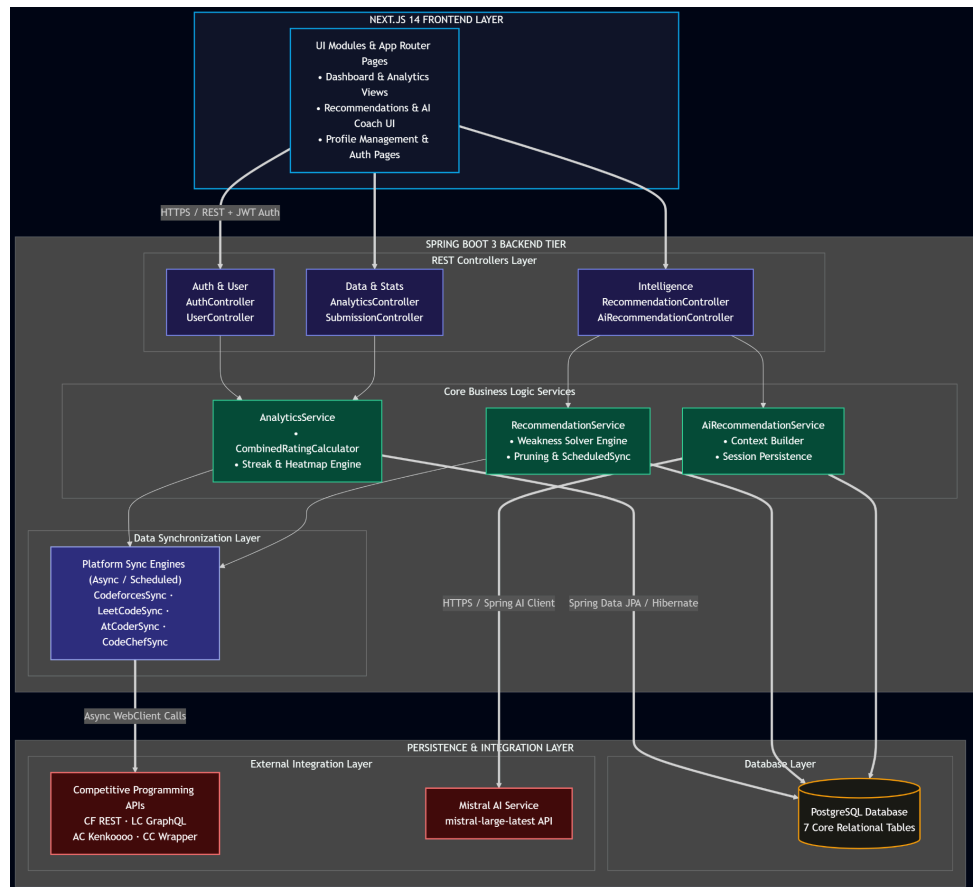


Figure 13: Component Diagram

Table 19: Layers and responsibilities

Tier / layer	Technology	Responsibility
Presentation	Next.js 14 (App Router), React, Tailwind CSS, Recharts, Framer Motion, Zustand, TanStack Query	Routing, forms, charts, client-side hidden-insights engine, JWT storage
Application - API	Spring MVC controllers (Auth, User, Analytics, Recommendation, AI, Submission, Admin)	HTTP contract, validation, DTO mapping, error translation
Application - business	Services: Analytics, Recommendation, AIRecommendation, Auth, User, Pruning	Rating, streaks, weakness scoring, recommendation selection, AI orchestration
Application - integration	CodeforcesDataService, LeetcodeDataService, AtCoderDataService, CodeChefDataService, DirectMistralService, OpenRouterService	Outbound calls to third-party APIs; isolated per provider
Security	Spring Security, JwtAuthenticationFilter, JwtUtil, UserDetailsServiceImpl	Stateless authentication and role-based authorisation
Data	Spring Data JPA / Hibernate, PostgreSQL 15	Persistence, queries with JOIN FETCH, bulk delete for pruning

6.2 Package Structure

The backend is organised by technical layer under `com.codepulse`:

```

com.codepulse
├── config          DataSeeder (20 topics), SecurityConfig, WebClientConfig,
RestTemplate beans
├── controller      Auth, User, Analytics, Recommendation, AIRecommendation,
Submission, Admin
├── dto             request/ (Login, Register, UpdateProfile, AiPrompt)
                    response/ (ApiResponse, AuthResponse, PerformanceAnalytics,
Recommendation, UserProfile, AiSession)
├── entity          User, Problem, Topic, Submission, Recommendation,

```

exception	AiRecommendationSession, AiRecommendationItem
repository	BadRequest, ResourceNotFound, GlobalExceptionHandler
security	one Spring Data repository per entity
UserPrincipal	JwtAuthenticationFilter, JwtUtil, UserDetailsServiceImpl,
service	interfaces + impl/ (Analytics, Auth, Recommendation, User,
AiRecommendation,	Codeforces/Leetcode/AtCoder/CodeChef data services, Pruning,
ScheduledSync,	CombinedRatingCalculator, GroqService, DirectMistralService,
OpenRouterService,	OpenRouterModelDiscovery, ModelHealthRegistry)

6.3 Design Classes of the Problem Domain

The domain model identifies the persistent entity classes and their relationships. Each entity maps one-to-one to a table described in Chapter 5.

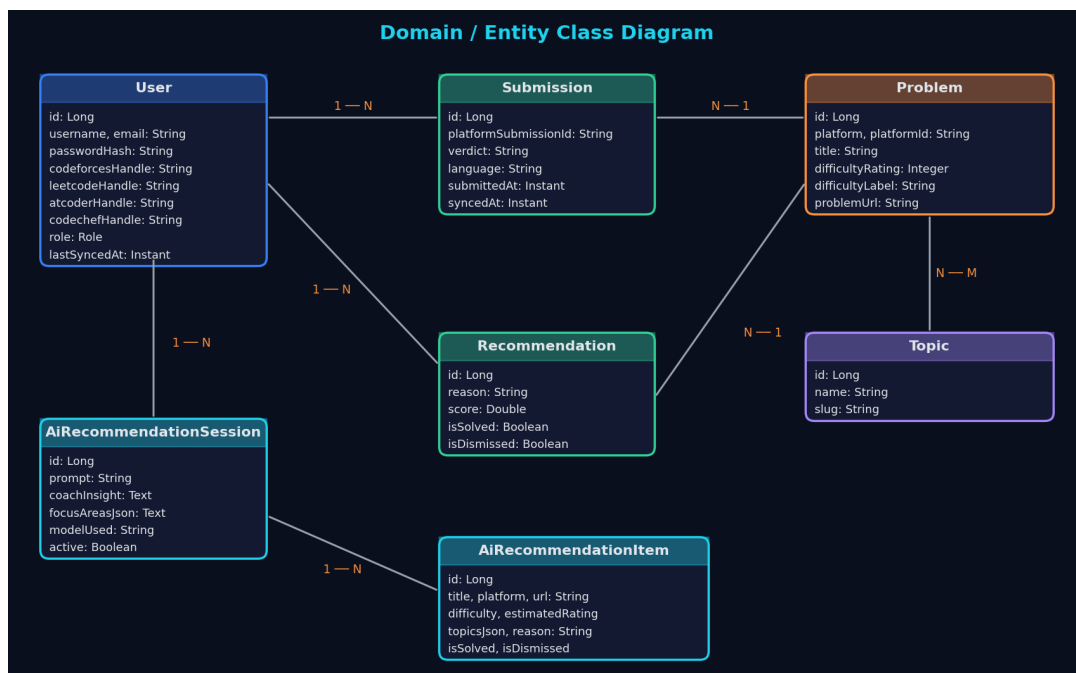


Figure 14: Domain (entity) class diagram

Table 20: Domain classes and responsibilities

Class	Responsibility	Key attributes / methods
User	Identity, credentials, role and the four platform handles	username, email, passwordHash, *Handle, role, lastSyncedAt
Problem	A problem on any platform with difficulty and topics	platform, platformId, title, difficultyRating, difficultyLabel, problemUrl
Topic	Algorithm tag; many-to-many with Problem	name, slug
Submission	One attempt by a user on a problem	verdict, language, submittedAt, syncedAt
Recommendation	Curated problem for a user with reason and score	score, isSolved, isDismissed
AiRecommendationSession	One AI coaching consultation	prompt, coachInsight, focusAreasJson, modelUsed, active
AiRecommendationItem	One recommended problem inside a session	title, platform, url, difficulty, topicsJson, reason

6.4 Service-Layer Design Classes

Business logic lives in stateless Spring services. Interfaces (`AnalyticsService`, `RecommendationService`, `AuthService`, `UserService`) decouple controllers from implementations, which makes mocking in unit tests straightforward.

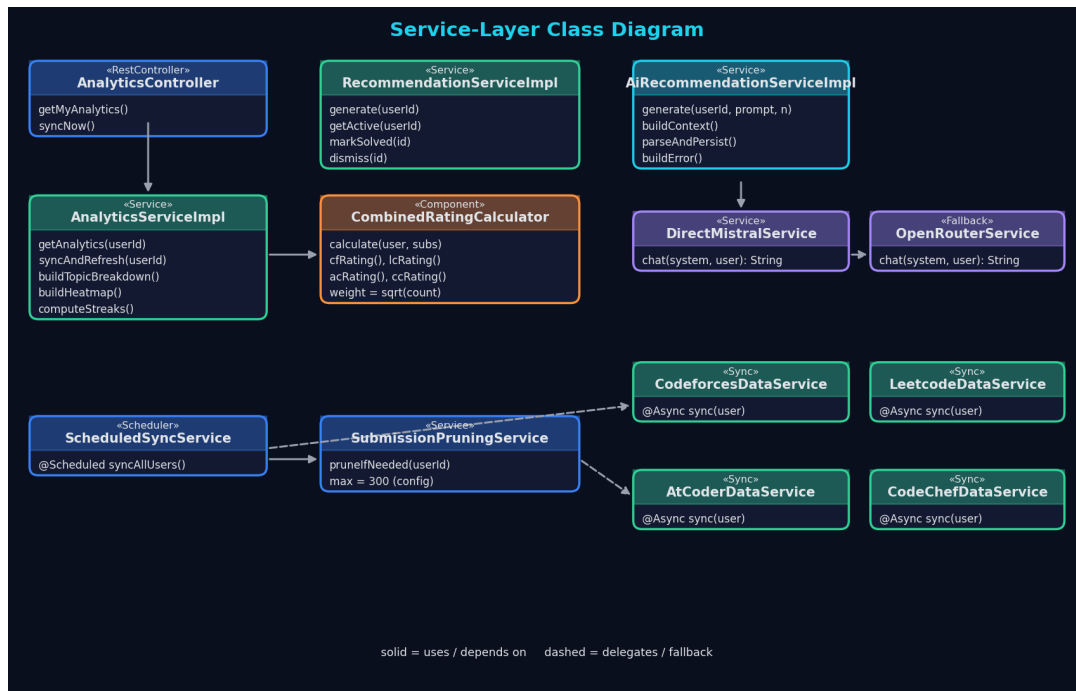


Figure 15: Service-layer class diagram

Table 21: Service-layer classes

Class	Responsibility	Key methods
<code>AnalyticsServiceImpl</code>	Computes all combined metrics, streaks, topic breakdown, heatmap, verdict distribution	<code>getAnalytics()</code> , <code>syncAndRefresh()</code>
<code>CombinedRatingCalculator</code>	Weighted Elo-like rating across four platforms	<code>calculate(user, submissions)</code>
<code>RecommendationServiceImpl</code>	Weakness-based algorithmic problem selection and feedback	<code>generate()</code> , <code>getActive()</code> , <code>markSolved()</code> , <code>dismiss()</code>
<code>AiRecommendationServiceImpl</code>	Orchestrates the AI Coach: builds context, orders providers, validates the answer, persists the session outside long transactions	<code>generateRecommendations()</code> , <code>getLatestSession()</code> , <code>persist()</code> , <code>buildError()</code>
<code>DirectMistralService</code>	Mistral provider: model discovery, best-first trial, one retry on 429/5xx	<code>chat(system, user, validator)</code>
<code>OpenRouterService</code>	OpenRouter provider: free models only, at most 10 attempts within 90 s	<code>chat(system, user, validator)</code>
<code>SubmissionPruningService</code>	Enforces the per-user submission ceiling	<code>pruneIfNeeded(userId)</code>
<code>ScheduledSyncService</code>	Triggers sync for all active users every 6 hours	<code>@Scheduled syncAllUsers()</code>
<code>CodeforcesDataService</code>	Codeforces <code>/user.status</code> synchronisation	<code>@Async syncUserSubmissions(user)</code>
<code>LeetcodeDataService</code>	LeetCode GraphQL <code>recentSubmissionList</code>	<code>@Async syncUserSubmissions(user)</code>
<code>AtCoderDataService</code>	AtCoder kenkoooo submissions	<code>@Async syncUserSubmissions(user)</code>
<code>CodeChefDataService</code>	CodeChef community API	<code>@Async syncUserSubmissions(user)</code>

Class	Responsibility	Key methods
<code>JwtUtil / JwtAuthenticationFilter</code>	Create, validate and apply JWT on each request	<code>generateToken()</code> , <code>validate()</code> , <code>doFilterInternal()</code>
<code>GroqService</code>	Groq provider: discovers models for the key, ranks them best-first, JSON-mode with plain-mode fallback	<code>chat(system, user, validator)</code>
<code>OpenRouterModelDiscovery</code>	Finds truly free OpenRouter models ("free", zero price, text, ≥ 8k context); refreshed every 6 h	<code>getFreeModels()</code> , <code>refresh()</code>
<code>ModelHealthRegistry</code>	In-memory memory of failing / working models and providers with cooldowns	<code>order()</code> , <code>markFailure()</code> , <code>markSuccess()</code> , <code>lastGoodProvider()</code>
<code>AiJsonUtil</code>	Shared helpers: <think>/fence stripping, balanced-brace JSON extraction, content extraction, key cleaning, timeouts	<code>extractRecommendationJson()</code> , <code>extractContent()</code> , <code>cleanKey()</code>
<code>AiDiagnosticsController</code>	Self-test endpoint for each AI provider	<code>GET /api/ai/health/{provider}</code>

6.5 Persistent Data Sources

6.5.1 Databases

- **Primary store:** PostgreSQL 15. All eight tables of Section 5.1 are JPA-managed entities; Hibernate `ddl-auto=update` creates or alters tables at start-up.
- **Seed data:** the `DataSeeder` bean inserts 20 algorithm topics (Dynamic Programming, Graphs, Binary Search, Greedy, Trees, ...) on first boot if the `topics` table is empty.
- **External read-only sources:** Codeforces, LeetCode, AtCoder and CodeChef APIs are treated as remote data sources; nothing is written back to them.
- **Files:** the system persists no user files. Configuration is read from `application.properties` and environment variables.

6.5.2 Repository Classes

Table 22: Repository queries

Repository	Notable queries
<code>UserRepository</code>	<code>findByEmail</code> , <code>findByUsername</code> , <code>existsByEmail</code> , <code>existsByUsername</code> , <code>findAllByActiveTrue</code>
<code>SubmissionRepository</code>	<code>findById</code> and <code>findRecentById</code> using <code>SELECT DISTINCT s FROM Submission s JOIN FETCH s.problem p LEFT JOIN FETCH p.topics</code> ; existence check by platform submission id; bulk delete of oldest rows
<code>ProblemRepository</code>	lookup by platform + platformId; unsolved problems in topic set within a difficulty range
<code>TopicRepository</code>	<code>findBySlug</code> ; seeding check
<code>RecommendationRepository</code>	<code>findActiveById</code> with JOIN FETCH on problem and topics; delete by user
<code>AiRecommendationSessionRepository / ItemRepository</code>	latest active session by user; deactivate all by user; <code>saveAll</code> for items

6.5.3 Lazy-Loading Strategy

With `spring.jpa.open-in-view=false`, the persistence session closes as soon as a repository method returns. Accessing a lazy association afterwards raises `LazyInitializationException`. The design addresses this in two layers: (1) repository queries load `Problem` and `Topic` data with `JOIN FETCH` in the same SQL round-trip, and (2) service and controller methods that traverse associations are annotated `@Transactional(readOnly = true)` as a safety net.

6.6 Behavioural Representations

6.6.1 Sequence - Analytics Retrieval

The sequence below shows how the analytics component collaborates with the security filter, repository and rating calculator.

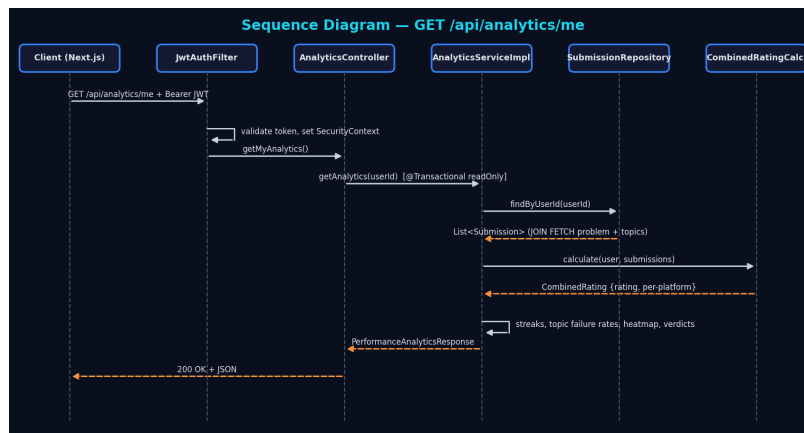


Figure 16: Sequence diagram - GET /api/analytics/me

6.6.2 Sequence - AI Recommendation

The AI Coach request is handled without holding a database transaction while the LLM runs. The controller delegates to AiRecommendationServiceImpl, which reads the context in a short read-only transaction, orders the providers through ModelHealthRegistry and asks each provider service in turn. A provider service discovers the usable models, calls them best-first and reports success or failure to the registry; the first answer that AiJsonUtil validates wins. The result is saved in a short write transaction. If every provider fails, buildError() returns a friendly response and the previous session is kept.

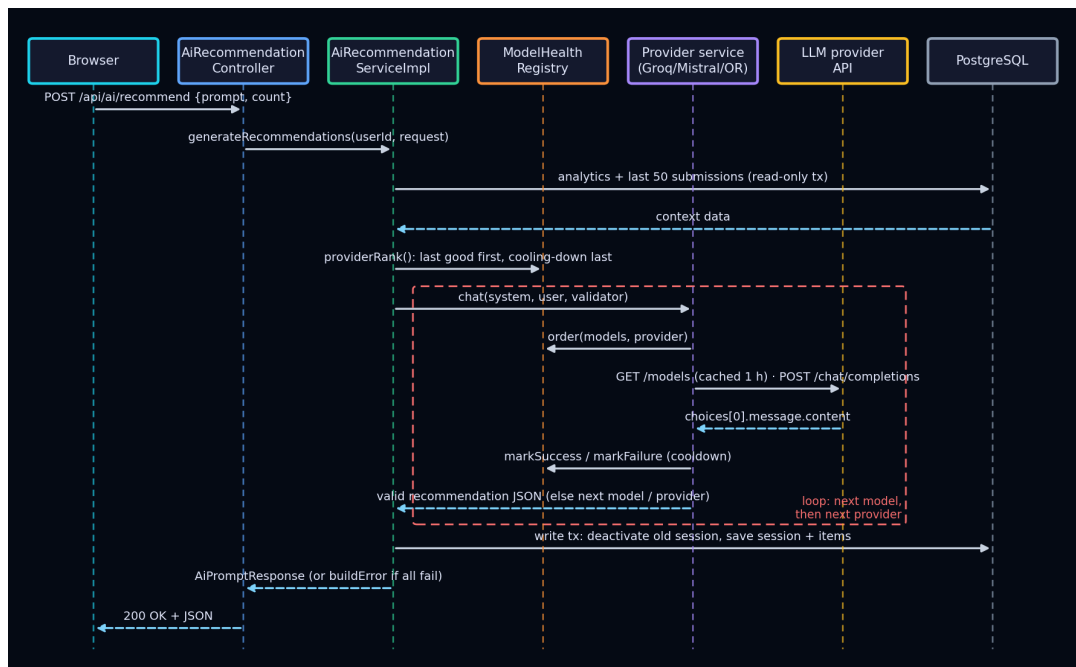


Figure 17: Sequence diagram - POST /api/ai/recommend

6.6.3 Lifecycle States of a Recommendation

Both curated recommendations and AI items share the same lifecycle:

Table 23: Recommendation lifecycle

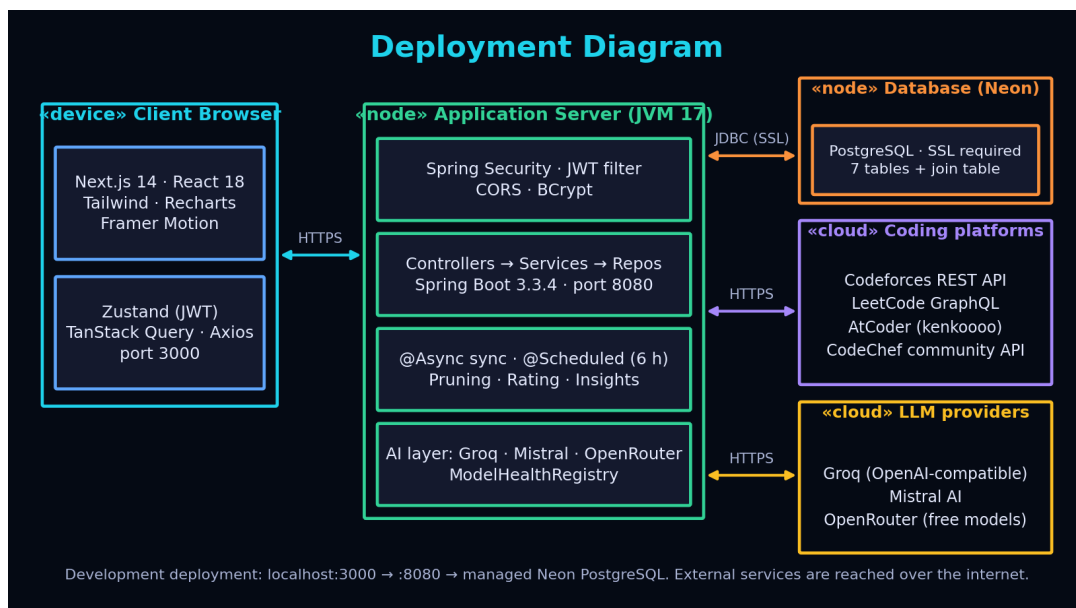
State	Entry event	Exit events	Visible to user
Active	Generated by engine / AI	Solve, dismiss, regenerate	Yes
Solved	PATCH .../solved	Regenerate (replaced)	No (removed from active list)
Dismissed	PATCH .../dismiss	Regenerate (replaced)	No
Replaced / inactive	New generation request	-	No (AI session <code>active = false</code>)

6.6.4 Behaviour of the Synchronisation Component

`ScheduledSyncService` iterates active users and invokes the four `@Async` services. Each service executes the same template: (1) read handle, (2) call remote API with a timeout, (3) map the response to `Problem`, `Topic` and `Submission` objects, (4) skip any submission whose `platformSubmissionId` already exists, (5) save the remainder. Exceptions are caught per platform, logged with the platform name, and never propagate to the scheduler. After all lanes complete, `SubmissionPruningService.pruneIfNeeded()` runs once per user.

6.7 Deployment Diagram

The deployment view elaborates the physical nodes, their runtime technology and communication paths. In development all three nodes run on one machine; in a production-like setup they can be separated without code changes because they communicate only through HTTP and JDBC.

**Figure 18:** Deployment diagram**Table 24:** Deployment nodes

Node	Runtime	Port	Protocol
Client browser	Next.js 14 application (dev server or built output)	3000	HTTPS / JSON to backend
Application server	JVM 17 running the Spring Boot 3.3.4 executable JAR	8080	REST; JDBC to DB; HTTPS to APIs
Database server	PostgreSQL (Neon-hosted, SSL) or local 15; database codepulse	5432	JDBC
External services	Codeforces, LeetCode, AtCoder (kenkoooo), CodeChef, Groq, Mistral AI, OpenRouter	443	HTTPS

6.8 Core Algorithms

6.8.1 CodePulse Rating

A single Elo-like number is computed from all connected platforms. Per-platform ratings are derived as follows:

Table 25: Per-platform rating formulas

Platform	Rating formula	Cap
Codeforces	Actual rating from <code>/user.rating</code> ; otherwise estimated from the average difficulty of solved problems	-
LeetCode	$1200 + (\text{easy} \times 2) + (\text{medium} \times 9) + (\text{hard} \times 22) + \text{acceptance-rate bonus}$	3200
AtCoder	Average difficulty of solved problems $\times 1.05$	-
CodeChef	$1000 + (\text{easy} \times 3) + (\text{medium} \times 10) + (\text{hard} \times 20) + (\text{expert} \times 35)$	3000

```
weight(platform) = sqrt(submissionCount(platform)) // diminishing returns
CodePulse Rating = SUM(rating_p * weight_p) / SUM(weight_p)
CodePulse Rating = clamp(CodePulse Rating, 800, 3800)
```

Using the square root of the submission count prevents a platform with very many submissions from dominating the result while still rewarding sustained activity. Platforms with no activity contribute zero weight, so a user with only one connected platform still receives a meaningful rating.

6.8.2 Topic Weakness Score

```
for each topic t:
    attempts(t) = number of submissions on problems tagged t
    failures(t) = attempts(t) - accepted(t)
    weakness(t) = failures(t) / attempts(t) // 0.0 strong ... 1.0 weakest
weakTopics = topics sorted by weakness desc (min. attempts threshold applied)
strongTopics = topics sorted by weakness asc and accepted(t) high
```

6.8.3 Streak Calculation

Submission timestamps from all platforms are reduced to a set of distinct calendar days. The **current streak** counts consecutive days ending today (or yesterday); the **longest streak** is the maximum run in the set; **total active days** is the set size. Because platforms are merged first, two Codeforces submissions on day 1 and one LeetCode submission on day 2 yield a streak of 2.

6.8.4 Curated Recommendation Selection

```
weak = top weakness topics (failure rate > 30%)
rating = estimated user rating (CodePulse Rating)
range = difficulty band slightly above the estimated rating // stretch zone
cand = problems WHERE topic IN weak AND difficulty IN range AND id NOT IN
solved(user)
picks = rank cand by (weakness(topic), closeness to rating); take up to 10
if |picks| < 10: fill with difficulty-matched problems from any topic
delete previous recommendations(user); save picks with reason + score
```

The failure-rate threshold and difficulty band are configuration constants in the service and can be tuned without changing the architecture.

6.8.5 Submission Pruning

After every sync the service counts the user's submissions. If the count exceeds `codepulse.submissions.max-per-user` (default 300), the surplus oldest rows (ordered by `submitted_at` ascending) are removed using a single bulk `DELETE ... WHERE id IN (...)` statement. This keeps database size bounded and analytics always based on the latest activity.

6.8.6 Hidden Insights Engine

The insights engine runs in the frontend on the analytics response, with no extra backend calls. It evaluates a rule set and emits up to ten findings in five categories.

Table 26: *Hidden insights rule set*

Category	Rule (example)	Finding
Critical	Failure rate on a topic > 55%	Critical weakness with exact percentage
Critical	0 accepted submissions on a topic	Blind spot detected
Warning	Acceptance rate < 35%	Submit more carefully
Warning	> 75% of solves are Beginner / Easy	Comfort-zone alert
Warning	Streak broken after a best of ≥ 7 days	Restart today
Warning	Activity down > 50% versus previous 30 days	Activity drop
Success	Current streak ≥ 14 days	Warrior recognition
Success	Acceptance rate > 80%	Sharpshooter
Tip	< 6 distinct topics despite > 25 solved	Explore more topics
Pattern	Top three topics with ≥ 15 solves each	Signature strengths

6.9 Design Alternatives and Refactoring

Each component-level design representation was reviewed for alternatives and refactored where a better option emerged. The table records the principal decisions.

Table 27: *Design alternatives considered*

Component	Alternatives considered	Decision and rationale
LLM integration	(a) Spring AI SDK; (b) direct REST with <code>RestTemplate</code> + <code>JsonNode</code>	(b). Spring AI milestone releases (M4–M6) create an internal <code>ObjectMapper</code> that cannot be customised. When Mistral added <code>prompt_tokens_details</code> to its response, every SDK call failed with <code>UnrecognizedPropertyException</code> . Parsing into a <code>JsonNode</code> tree ignores unknown fields.
Lazy loading	(a) Open-session-in-view; (b) <code>JOIN FETCH</code> + <code>@Transactional(readOnly)</code>	(b). OSIV hides N+1 queries and holds connections during view rendering. Explicit fetching gives one SQL round-trip per request and predictable performance.
Platform sync	(a) One monolithic sync class; (b) one service per platform	(b). Failures are isolated, platforms run in parallel and adding a platform needs only one new class (NFR-6).
Rating weight	(a) Arithmetic mean; (b) weight by submission count; (c) weight by $\sqrt{\text{count}}$	(c). Mean ignores effort; raw count lets one platform dominate; $\sqrt{}$ gives diminishing returns.
Data growth	(a) Keep all history; (b) cap per user	(b). A 300-row cap bounds storage and keeps analytics responsive (NFR-1, NFR-4).
Insights engine	(a) Backend computation; (b) client-side rules	(b). Rules need only the analytics response; client-side evaluation avoids extra API calls and keeps the backend focused on data.
LLM availability	(a) Single provider; (b) primary + OpenRouter fallback; (c) ordered multi-provider chain with run-time model discovery	(c). Free tiers are rate-limited and models are retired without notice; a chain of three providers with failure memory keeps the AI Coach available.
Sync trigger	(a) Only manual; (b) manual + 6-hour cron	(b). Fresh data without user action; manual option for immediacy.
Authentication	(a) Server sessions; (b) stateless JWT	(b). Fits the SPA frontend and keeps the backend horizontally scalable.
Model configuration	(a) Fixed model names in properties; (b) discover models per API key at run time	(b). Model names get decommissioned (HTTP 404); discovery needs only the API key and adapts automatically.

Component	Alternatives considered	Decision and rationale
Reasoning models	(a) Fall back to the reasoning field; (b) ignore it and require message content	(b). Reasoning text is not the answer and caused "no valid JSON" failures; such models are skipped.
Transaction scope of the AI call	(a) One @Transactional method; (b) read transaction, LLM call outside any transaction, short write transaction	(b). A slow model no longer holds a database connection for minutes.
Failure handling	(a) Retry the same model; (b) cooldown per model/provider in ModelHealthRegistry	(b). Later requests skip known-bad models and stay fast; nothing is banned permanently.

Refactorings applied during development: extraction of CombinedRatingCalculator out of AnalyticsServiceImpl; replacement of the SDK-based AI client by direct provider services (DirectMistralService, GroqService, OpenRouterService); extraction of ModelHealthRegistry, OpenRouterModelDiscovery and AiJsonUtil from the provider classes; moving the LLM call out of any database transaction; introduction of SubmissionPruningService as a separate component; moving the JOIN FETCH logic into repository queries; and consolidating DTO construction into the service layer.

7. Interface Design

The interface was designed from the user-interface analysis of the primary users (competitive programmers and students) who want quick insight with minimal clicks. The visual language is a dark "cyberpunk" theme with neon accent colours, glassmorphism cards and animated transitions, implemented with Tailwind CSS and Framer Motion.

7.1 Design Principles

- **Glanceability:** the dashboard exposes four headline numbers (rating, solved, streak, acceptance) before any chart.
- **No duplication:** the Dashboard summarises; the Analytics page goes deep.
- **Feedback for every action:** skeleton loaders, spinners, toasts and empty states with a call to action (NFR-5).
- **Direct access to the source:** every problem title and platform handle is a link that opens the original page in a new tab.
- **Responsive layout:** sidebar navigation collapses on narrow screens.

7.2 Pages and Routes

Table 28: Pages, routes and UI objects

Route	Purpose	Key UI objects
/auth/login	Authenticate and receive a JWT	Email field, password field with visibility toggle, submit button, typing-animation terminal hint, link to register
/auth/register	Create an account (2-step wizard)	Step 1: username and full name. Step 2: email and password. Progress bar, Back / Next buttons
/dashboard	Combined overview	Four StatCards, StreakWidget, Platform Connection Grid, quick-action tiles, Recent Submissions timeline, Sync button
/analytics	Deep analysis	Insights panel, platform comparison bar chart, CF rating area chart, difficulty bar, weakness radar, verdict donut, topic coverage bars, activity heatmap
/recommendations	Curated problems	Platform filter tabs, problem cards with link, Solve / Dismiss buttons, score bar, Generate New button
/ai-recommend	AI Coach	Prompt textarea, example chips, count selector (4 / 6 / 8), Ask AI button, Coach Insight card, focus-area chips, AI cards

Route	Purpose	Key UI objects
<code>/profile</code>	Account management	Avatar, connection-status badges, four handle inputs pre-filled from server, external profile links, Save button

7.3 User-Interface Objects and Actions

Table 29: *Interface objects and operations*

UI object	Description	Action (operation)	Resulting change
StatCard	Animated number tile with neon border and LIVE badge	On mount: CountUp from 0 to the value over 2 s	Number displayed
Platform grid cell	Handle, solved count and acceptance rate, or "Add handle →"	Click handle / link	Opens the platform profile in a new tab or navigates to <code>/profile</code>
Sync button	Triggers manual synchronisation	Click → <code>POST /api/analytics/sync</code>	Spinner shown; after ~4 s the query cache is invalidated and data refreshes
Submission row	Platform badge, title, topic chip, verdict badge, time-ago	Click title	Opens <code>problemUrl</code> in a new tab
Insight card	Coloured card by category with icon, title, text	Appear after data load	Staggered Framer Motion animation
AI prompt form	Textarea, chips, count selector, submit	Submit / Ctrl+Enter / click chip	POST; generating state; session replaced on success
Recommendation card	Platform badge, difficulty chip, title, topics, reason, score bar	Solve / Dismiss	<code>PATCH</code> call; card exits with animation
Profile form	Name, avatar URL, four handles	Save Changes → <code>PUT /api/users/me</code>	Toast confirmation; handles propagate to dashboard
Platform filter tabs	All / CF / LC / AC / CC	Click tab	List filtered client-side

7.4 Events and State Changes

Table 30: *UI events and state transitions*

Event (user action)	From state	To state
Open /dashboard	None	Loading skeleton
Profile + analytics loaded	Loading skeleton	Data loaded
Loaded but no handles saved	Loading skeleton	Empty state (prompt to add handles)
Click Sync All	Data loaded	Syncing (spinner)
Sync timeout elapses, cache invalidated	Syncing	Data loaded (refreshed)
Submit AI prompt	No session / Session displayed	Generating (animated dots)
AI response 200 with session	Generating	Session displayed
AI error	Generating	Error message with previous session retained
Click Solve / Dismiss	Session / list displayed	Item fades out; list updated
Token missing / expired (401)	Any page	Redirect to <code>/auth/login</code>

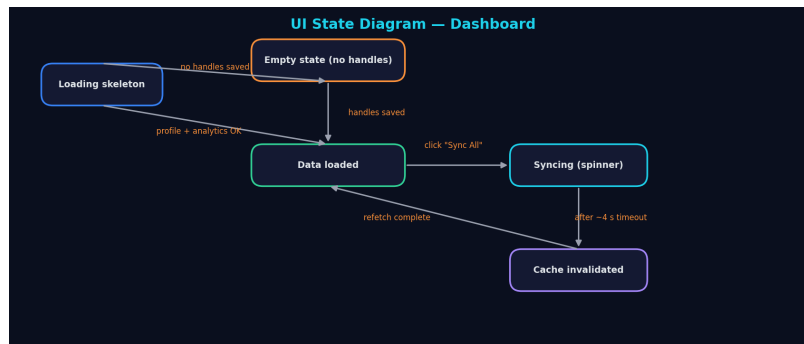


Figure 19: State diagram - Dashboard

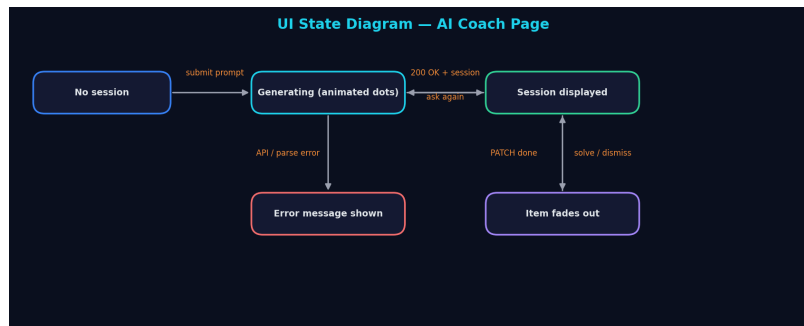


Figure 20: State diagram - AI Coach page

7.5 Screen Designs

The following screens are screenshots of the running application and depict each interface state as the end user sees it.

7.5.1 Authentication

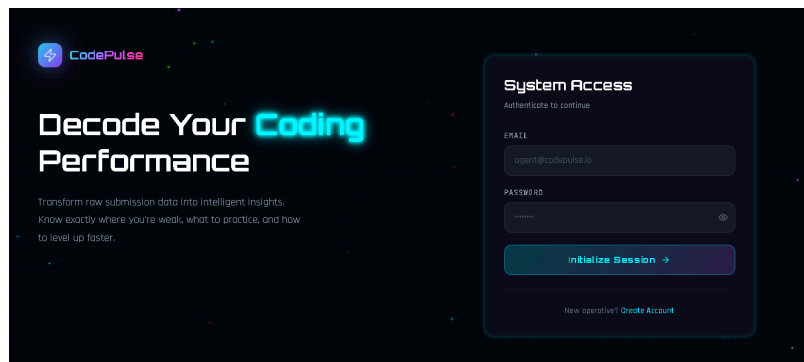


Figure 21: Login and two-step registration screens

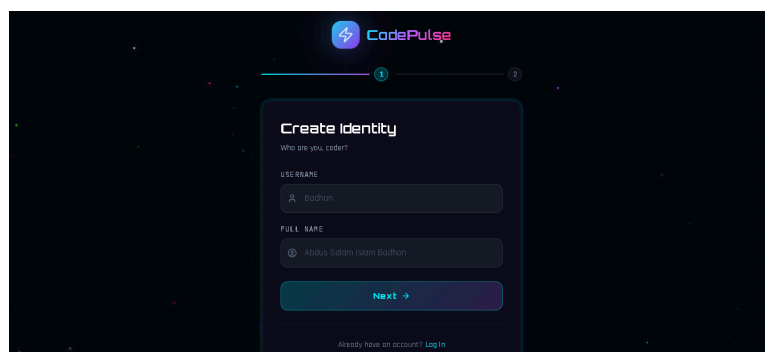


Figure 22: Registration step 1: username and full name

Set Credentials
Secure your account

EMAIL
badhon201920@gmail.com

PASSWORD
[Masked]

Back Launch

Already have an account? Log In

Figure 23: Registration step 2: email and password

7.5.2 Dashboard



Figure 24: Dashboard screen

7.5.3 Analytics

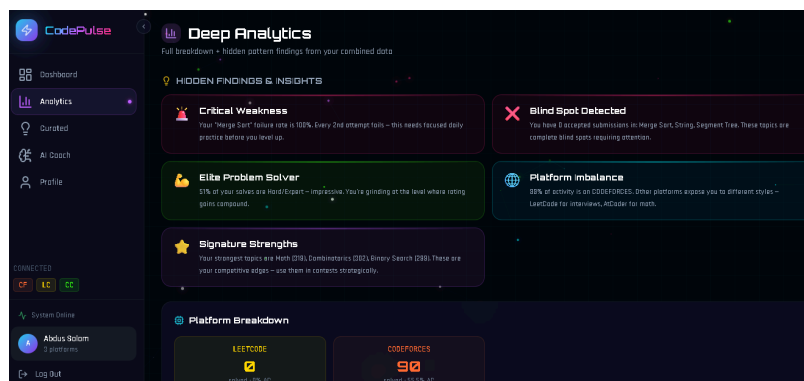


Figure 25: Analytics screen

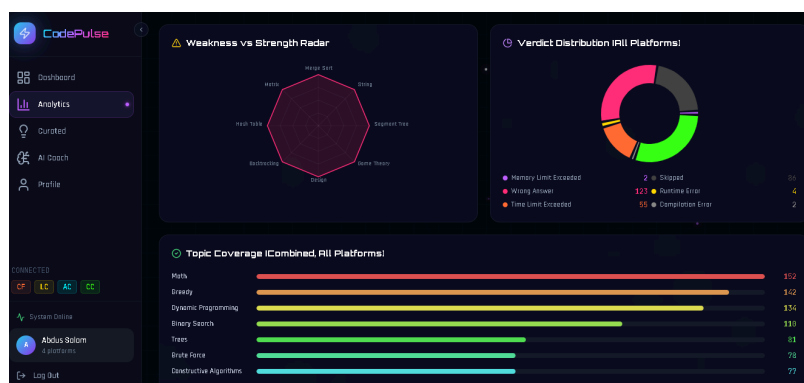


Figure 26: Analytics: weakness radar, verdict distribution and topic coverage

7.5.4 Curated Recommendations

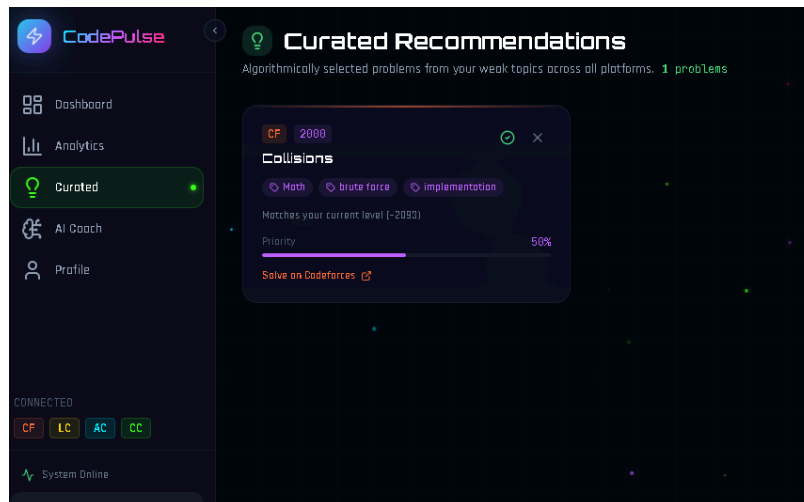


Figure 27: Recommendations screen

7.5.5 AI Coach

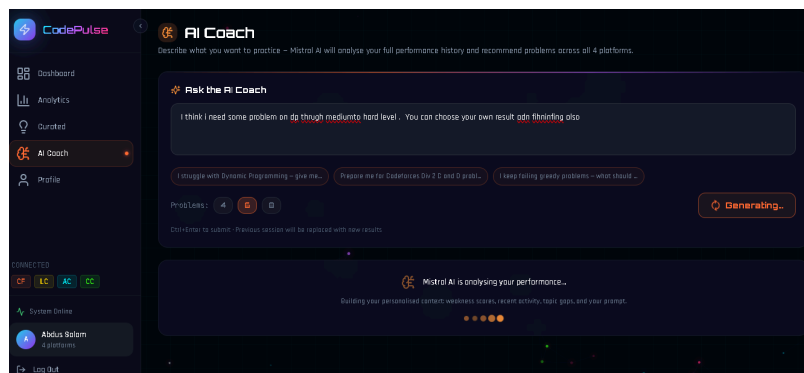


Figure 28: AI Coach screen

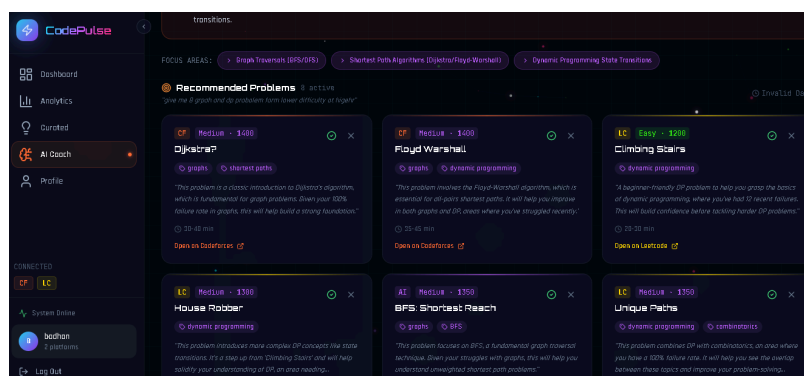


Figure 29: AI Coach: recommended problems with focus areas and reasons

7.5.6 Profile

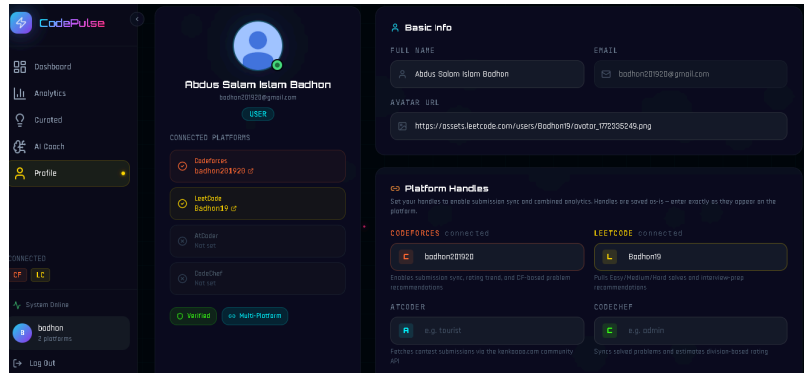


Figure 30: Profile screen

8. Implementation

8.1 Technology Stack

Table 31: Technology stack

Layer	Technology	Version / notes
Language (backend)	Java	17 (LTS)
Framework (backend)	Spring Boot	3.3.4 (spring-boot-starter-parent)
Web / validation	spring-boot-starter-web, spring-boot-starter-validation	REST controllers, bean validation
Security	Spring Security + jjwt	JWT HS256 (jjwt 0.11.5), BCrypt password encoder
ORM	Spring Data JPA / Hibernate	ddl-auto=update, open-in-view disabled
Database	PostgreSQL	15+
HTTP clients	Spring WebFlux <code>WebClient</code> ; <code>RestTemplate</code>	<code>WebClient</code> for platform APIs; <code>RestTemplate</code> for LLM calls
AI providers	Groq · Mistral AI · OpenRouter over direct HTTP	Models auto-discovered per key, best-first
AI resilience	ModelHealthRegistry + AiJsonUtil	Cooldowns, strict JSON validation, fallback
Utilities	Lombok	Boilerplate reduction for entities and DTOs
Build	Maven	3.9+
Frontend	Next.js 14 (App Router), TypeScript, React	Server and client components
Styling / animation	Tailwind CSS, Framer Motion	Custom neon theme; page reveals and card exits
Charts	Recharts	Area, bar, pie / donut and radar charts
Client state	Zustand + TanStack Query v5	Auth store; server-state cache with invalidation
Tooling	Git / GitHub, Postman, IntelliJ / VS Code	Repository: github.com/Badhon1401/SPL-3

8.2 Backend Configuration

All environment-specific values live in `application.properties`; secrets should be supplied by environment variables in any shared deployment.

Table 32: Key configuration properties

Property	Purpose	Example
<code>spring.datasource.url</code>	JDBC URL of the PostgreSQL database	<code>jdbc:postgresql://localhost:5432/codepulse (Neon: ...?sslmode=require)</code>
<code>spring.datasource.username / password</code>	Database credentials	<code>postgres / *secret*</code>
<code>spring.jpa.hibernate.ddl-auto</code>	Schema management	<code>update</code>
<code>spring.jpa.open-in-view</code>	Disable OSIV	<code>false</code>
<code>jwt.secret</code>	Base64-encoded 256-bit signing key	<code>openssl rand -base64 32</code>
<code>jwt.expiration</code>	Token lifetime in ms	<code>86400000 (24 h)</code>
<code>app.groq.api-key</code>	Groq API key (optional if another provider is configured)	<code>*secret*</code>
<code>app.openrouter.api-key</code>	OpenRouter API key (optional)	<code>*secret*</code>
<code>codepulse.submissions.max-per-user</code>	Pruning ceiling	<code>300</code>
<code>mistral.api-key</code>	Mistral AI key (alias <code>spring.ai.mistralai.api-key</code>)	<code>*secret*</code>
<code>app.openrouter.models</code>	Optional override list; empty = automatic discovery of free models	<code>(empty)</code>
<code>app.openrouter.max-attempts / total-timeout-seconds</code>	Attempts and time budget for the OpenRouter stage	<code>10 / 90</code>
<code>codepulse.sync.enabled / codepulse.sync.cron</code>	Scheduled synchronisation switch and schedule	<code>true / 0 0 */6 * * *</code>

8.3 Security Implementation

Security is implemented with Spring Security in a **stateless** configuration. The flow on each request is:

1. The `JwtAuthenticationFilter` reads the `Authorization: Bearer <token>` header.
2. `JwtUtil` verifies the signature and expiry and extracts the user identity.
3. `UserDetailsServiceImpl` loads the user and wraps it in a `UserPrincipal` that carries the role.
4. The authenticated principal is placed in the `SecurityContext`.
5. `SecurityConfig` permits `/api/auth/**`, restricts `/api/admin/**` to `ROLE_ADMIN` and requires authentication for everything else; CORS is configured for the frontend origin.

Table 33: Security controls

Concern	Mechanism
Password storage	BCrypt hash; plaintext never stored or logged
Session management	Stateless; no server session; token in client store
Authorisation	Role claim; <code>@PreAuthorize</code> / URL rules for admin endpoints (403 for <code>ROLE_USER</code>)
Input validation	Bean Validation on request DTOs; <code>GlobalExceptionHandler</code> maps errors to <code>ApiResponse</code> with proper status codes
Secrets	JWT secret and API keys supplied via properties / environment, never committed
Data access	Parameterised JPA queries; no string-built SQL

8.4 Data Synchronisation Implementation

Each platform has its own `@Async` service. The common template is shown as pseudocode:

```
@Async syncUserSubmissions(user) :
    handle = user.<platform>Handle ; if handle is blank: return
    try:
```

```

raw = http GET/POST platform API (limit 200, timeout)
for item in raw:
    if submissionRepo.exists(platform, item.id): continue // dedup (FR-6)
    problem = problemRepo.findOrCreate(platform, item.problem) // + topics
    submissionRepo.save(new Submission(user, problem, item.verdict,
item.time))
    catch (Exception e):
        log.error("Sync failed for {}: {}", platform, e) // isolate
failure (NFR-3)

```

Platform-specific notes:

- **Codeforces:** `/user.status?handle=...&from=1&count=200`; verdicts such as `OK`, `WRONG_ANSWER` and `TIME_LIMIT_EXCEEDED` are normalised to the internal verdict names. Problem tags become topics; the numeric problem rating becomes `difficultyRating`. `/user.rating` supplies the rating trend.
- **LeetCode:** GraphQL query for recent submissions; the Easy / Medium / Hard label is stored as `difficultyLabel` because no numeric rating is exposed.
- **AtCoder:** `kenkoooo` community API for submissions and problem difficulty estimates.
- **CodeChef:** community wrapper API; difficulty is mapped to bands used by the rating formula.

`ScheduledSyncService` uses a Spring `@Scheduled` cron expression with a 6-hour period to synchronise all active users. The manual endpoint `POST /api/analytics/sync` calls the same services and returns immediately because the work runs asynchronously.

8.5 Analytics Implementation

`AnalyticsServiceImpl.getAnalytics()` is annotated `@Transactional(readonly = true)`. It loads submissions once and derives every metric in memory, which is efficient for the bounded 300-row dataset. The returned `PerformanceAnalyticsResponse` contains:

Table 34: Analytics response structure

Field group	Content
Totals	totalSubmissions, acceptedSubmissions, acceptanceRate, uniqueProblemsSolved
Rating	CodePulse Rating, tier, per-platform ratings
Streaks	currentStreak, longestStreak, activeDays
Topic data	topicBreakdown (accepted per topic), weaknessScores (0.0–1.0 per topic)
Difficulty	difficultyBreakdown: Beginner / Easy / Medium / Hard / Expert
Activity	activityHeatmap: map of ISO date → submission count (last 365 days)
Verdicts	verdictDistribution: ACCEPTED, WRONG_ANSWER, TLE, ...
Platform data	platformBreakdown: per-platform solved count, acceptance rate, verdicts, topic coverage
Trend	ratingTrend: list of {date, rating} from Codeforces

8.6 Recommendation Implementation

`RecommendationServiceImpl.generate()` runs the selection algorithm of Section 6.8.4 inside a transaction: it deletes the user's previous recommendations, selects up to ten unsolved problems and persists them with a human-readable `reason` (for example "Targets your weak topic: Dynamic Programming") and a priority `score`. `markSolved` and `dismiss` simply flip boolean flags, so an item disappears from the active list but remains for audit and future feedback weighting.

Algorithm details (RecommendationServiceImpl). A topic is considered weak when its failure ratio exceeds 0.30. For the top three weak topics, up to three unsolved problems are selected whose difficulty lies between the estimated rating minus 200 (never below 800) and the estimated rating plus 300; the estimate is the mean difficulty of accepted problems (default 1200). Remaining slots, up to ten in total, are filled with unsolved problems in the same band with score 0.5 and the reason "Matches your current level". The generating method

is transactional, and the read paths (analytics, recommendations) use `@Transactional(readOnly = true)` together with JOIN FETCH queries so that Problem and Topic data are initialised before the session closes.

Scheduled synchronisation. `ScheduledSyncService` runs every six hours (cron `0 0 */6 * * *`, configurable; switch `codepulse.sync.enabled`) and synchronises every active user who has at least one platform handle. Each user is processed inside its own try/catch so one failing account never stops the batch.

8.7 Frontend Implementation

- **Routing:** Next.js App Router pages for each route in Section 7.2; an authenticated layout renders the sidebar and redirects to login on a 401.
- **State:** a Zustand store holds the JWT and basic user info; TanStack Query caches server data (profile, analytics, recommendations, AI session) and is invalidated after sync, solve or dismiss actions.
- **API client:** a thin wrapper adds the `Authorization` header and normalises errors to toast notifications.
- **Charts:** Recharts components for platform comparison, rating trend, difficulty, weakness radar, verdict donut; a custom grid for the activity heatmap.
- **Hidden insights:** a pure function that maps the analytics response to an ordered list of finding objects (category, icon, title, text).
- **Motion:** Framer Motion for staggered insight cards and `AnimatePresence` exits when items are solved or dismissed.

The frontend source is organised by feature, as shown in Figure 31. The app directory holds the route groups ai-recommend, analytics, auth (login and registration), dashboard, profile and recommendations, each with its layout and page; components contains animations (ParticleBackground), charts (ActivityHeatmap, DifficultyBar, RatingLineChart, VerdictDonut, WeaknessRadar), dashboard widgets (RecommendationCard, StatCard, StreakWidget) and layout components (Providers, Sidebar); lib contains the Axios API client (api.ts), the rating-tier helper (ratingTier.ts) and the Zustand store (store.ts).

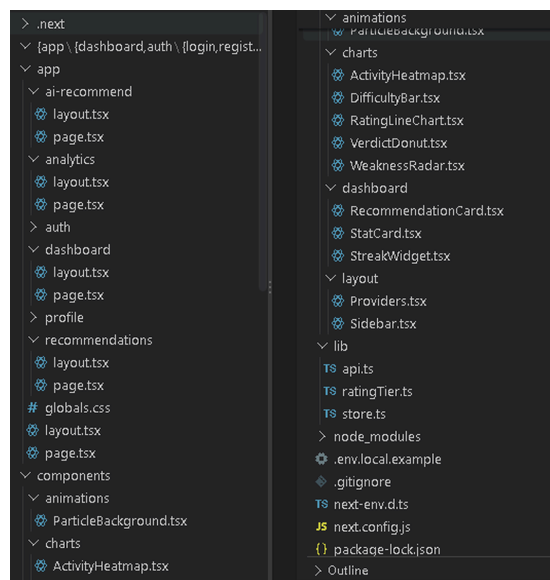


Figure 31: Frontend project structure (Next.js app router)

8.8 Implementation Challenges and Resolutions

Table 35: Implementation problems and fixes

Problem	Symptom	Resolution
Spring AI SDK incompatibility	<code>UnrecognizedPropertyException</code> on every AI call after the provider added <code>prompt_tokens_details</code>	Replaced the SDK with <code>DirectMistralService</code> using <code>RestTemplate</code> and <code>JsonNode</code>

Problem	Symptom	Resolution
<code>LazyInitializationException</code>	HTTP 500 on analytics, recent submissions and recommendations	<code>JOIN FETCH</code> in repository queries plus <code>@Transactional(readOnly = true)</code> on service / controller methods
LLM output with Markdown, reasoning text or truncation	JSON parse failures; "No valid JSON object"	<code>AiJsonUtil</code> strips <code><think></code> blocks and fences, scans for the first balanced JSON object with a non-empty recommendations array and treats anything else as a failure; reasoning content is never used as the answer.
Platform rate limits	Occasional empty sync from LeetCode	Per-platform error isolation; retry on next scheduled run or manual sync
Stale handles on dashboard	Handles missing after profile save	Profile and dashboard always fetch the profile from the server and invalidate the cache after save (FR-4)
Unbounded data growth	Slower analytics over time	<code>SubmissionPruningService</code> with 300-row ceiling
Free-tier rate limits and retired models	HTTP 429 or 404 from a model; long waits	<code>ModelHealthRegistry</code> puts failing models on a cooldown, models are discovered per key, and the chain continues with the next model or provider.
Reasoning models returning no answer	Empty content with <code>finish_reason=length</code>	Non-reasoning models are ranked first; empty content is a failure and the next model is tried.
Slow model holding a DB connection	Connection pool exhaustion during AI calls	The LLM call was moved outside any transaction; short read and write transactions only; explicit connect/read timeouts.
Keys with stray quotes or spaces	HTTP 401 from every provider	<code>AiJsonUtil.cleanKey()</code> trims whitespace and quotes; a 401 stops that provider immediately with a clear message.

9. AI Engineering Design

9.1 Goals

The AI Coach lets a user describe a practice goal in plain language - "I keep failing DP problems; give me a progression from easy to hard" - and receive problems chosen **for that person**, not generic lists. The LLM is therefore never asked a context-free question: it always receives the user's measured performance profile.

9.2 Pipeline

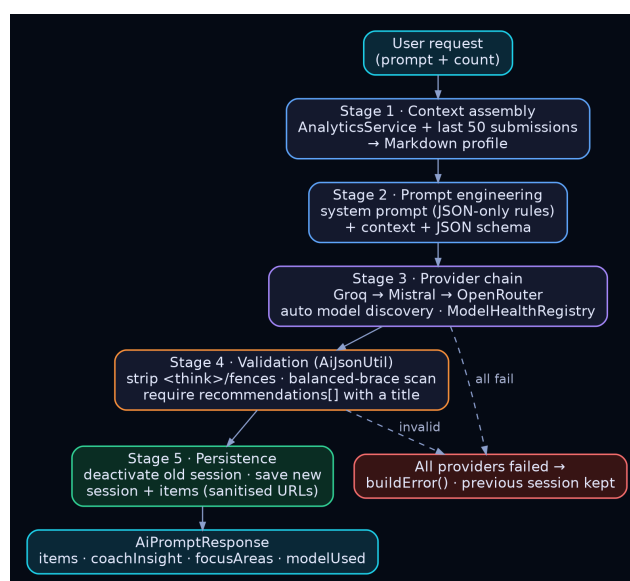


Figure 32: AI pipeline flow

Table 36: AI pipeline stages

Stage	Action	Output
1. Context assembly	Collect rating, per-platform stats, top weak and strong topics and recent activity in a short read-only transaction	Markdown context string
2. Prompt engineering	Combine the system prompt (JSON-only rules), context, user prompt and JSON schema; clamp the count to 1–10	System + user messages
3. Provider chain	Order providers (last good first); per provider discover usable models and call them best-first with timeouts; record success or failure in ModelHealthRegistry	Raw model content
4. Parsing and validation	AiJsonUtil strips noise, finds the first balanced JSON object and requires a non-empty recommendations array with titles	Validated JSON or failure (next model/provider)
5. Persistence	Sanitise URLs; deactivate the previous session; save the new session and items in a short write transaction	Session DTO returned

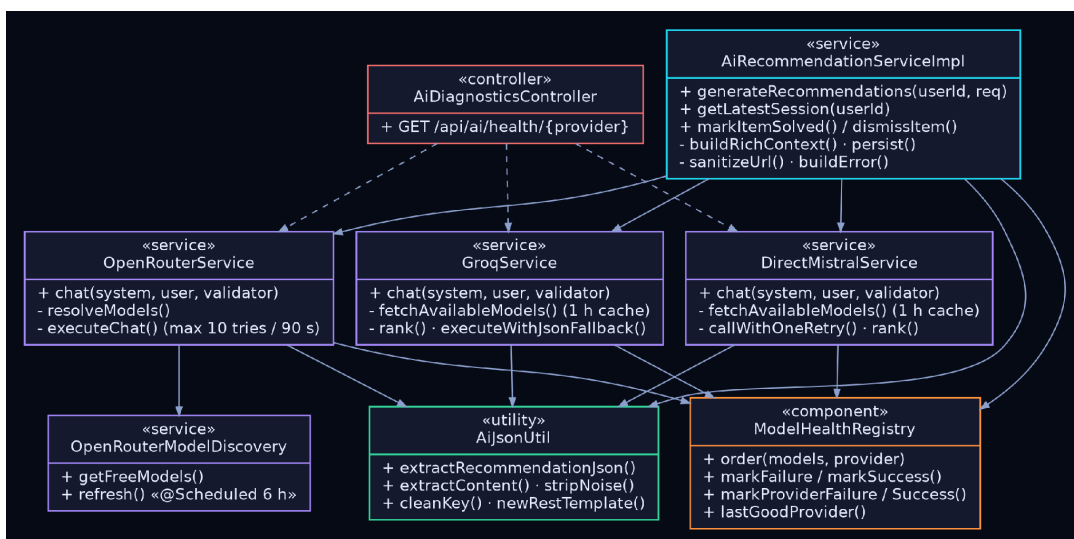
9.3 Provider and Model Selection

Table 37: Model selection

Provider	Role	Model discovery and ranking	Limits
Groq	First choice when configured	GET /models; excludes whisper, guard, tts, playai, orpheus, compound, embed, safeguard; rank: gpt-oss-120b, 70b, gpt-oss-20b, llama-4, qwen, kimi, 8b; at most 6 models	10 s connect / 45 s read; max_tokens ≥ 4096; JSON mode with plain-mode retry
Mistral AI	Second choice	GET /v1/models; excludes embed, ocr, moderation, voxtral, transcription, pixtral; rank: preferred (mistral-small-latest), small, nemo, ministral, medium, large; at most 5 models	10 s / 60 s; one retry after 1.5 s on 429/5xx; two consecutive 429 end the provider early
OpenRouter	Last resort	Only ids ending :free with zero price, text output, context ≥ 8k; non-reasoning first, JSON-capable first, larger context first; refreshed every 6 h; openrouter/free as final option	10 s / 45 s; at most 10 attempts and 90 s in total

9.4 AI Provider Layer: Design Classes

Figure 33 shows the classes of the AI provider layer. AiRecommendationServiceImpl is the only class used by the controller; it depends on the three provider services, on ModelHealthRegistry for ordering and on AiJsonUtil for validation. OpenRouterService additionally uses OpenRouterModelDiscovery, and AiDiagnosticsController exposes a self-test per provider.

**Figure 33:** Class diagram of the AI provider layer

9.5 Failure Memory and Fallback Strategy

ModelHealthRegistry is an in-memory, thread-safe memory of what works. A failing model or provider is put on a cooldown so the next request skips it instead of wasting up to a minute on it again; the model and provider that last succeeded are tried first. Nothing is banned permanently: when every candidate is cooling down, all are tried anyway. Table 38 lists the cooldowns used by the implementation.

Table 38: Cooldowns applied by ModelHealthRegistry

Event	Groq	Mistral AI	OpenRouter
Unusable output (no valid JSON)	10 min	10 min	30 min
HTTP 429 (rate limit)	3 min	3 min	3 min
HTTP 5xx	2 min	2 min	60 min
Other 4xx (400, 402, 403, 404)	60 min	60 min	60 min
Network or parse exception	5 min	5 min	30 min
HTTP 401 (bad key)	Provider stops immediately	Provider stops immediately	Provider stops immediately
Whole provider failed (chain level)	2 min	2 min	2 min

9.6 Response Validation (AiJsonUtil)

A provider response counts as a success only when it contains a JSON object with a non-empty recommendations array in which at least one item has a title. `extractRecommendationJson()` removes `<think>` blocks and Markdown fences, then scans for balanced braces (ignoring braces inside strings) and returns the first block that validates; truncated JSON is detected because its braces never balance. `extractContent()` reads `choices[0].message.content` only; the reasoning field is deliberately ignored, because reasoning text is thinking, not the answer. `cleanKey()` removes stray quotes and spaces from API keys and `newRestTemplate()` creates RestTemplates with real connect and read timeouts.

10. Testing

10.1 Testing Approach and Strategy

10.1.1 Objectives

- Validate the authentication, profile, analytics, recommendation, AI Coach, access-control and synchronisation endpoints end to end against the running application.
- Confirm that analytics, recent-submission and recommendation endpoints never return HTTP 500 or a `LazyInitializationException` (open-in-view disabled).
- Confirm that the AI pipeline answers through the provider chain and returns a graceful, size-limited result.
- Verify JWT protection and role-based access.

10.1.2 Levels of Testing

Table 39: Testing levels

Level	Scope	Tools
API / end-to-end	17 automated test cases against the running application (random port, real database, real AI provider)	JUnit 5, Spring Boot Test, TestRestTemplate, Maven Surefire
Regression	Analytics, recent-submission and recommendation endpoints re-checked after the JOIN FETCH fix (TC-08 to TC-11)	Same automated suite
System / acceptance	Screens captured in Chapter 7 and the scenarios of the user manual (Chapter 11)	Manual walk-through in Chrome

10.1.3 Test Environment

Table 40: *Test environment*

Item	Configuration
Backend	Java 17.0.6, Spring Boot 3.3.4, Maven, Windows; embedded Tomcat on a random port
Database	PostgreSQL hosted on Neon (SSL), accessed through HikariCP
Frontend	Node.js 18+, Next.js 14, port 3000 (manual UI checks)
Browsers	Google Chrome and Mozilla Firefox (latest)
External	Live Codeforces API (TC-17); Groq answered all AI requests (TC-14 to TC-16), with Mistral AI and OpenRouter in the provider chain as fallbacks

10.2 Item Pass / Fail Criteria

Table 41: *Pass / fail criteria*

Verdict	Criteria
PASS	The API returns the expected HTTP status and JSON structure; database state matches expectation; UI behaves as specified; no exception stack trace is returned to the client.
FAIL	Any HTTP 500; any <code>LazyInitializationException</code> in logs; incorrect data; unhandled AI provider error; duplicate or un-pruned submissions; protected endpoint reachable without a valid token.
Suspension	Testing of a feature is suspended if the database is unreachable or the build fails; resumes after the blocker is fixed.

10.3 Risks and Contingencies

Table 42: *Risks and contingencies*

Risk	Likelihood	Impact	Contingency
LeetCode GraphQL rate-limits	Medium	Medium	Retry later with back-off; test with pre-seeded database data
AtCoder kenkoooo API unavailable	Low	Low	Catch and log; other platforms continue; pre-seeded AtCoder submissions
AI provider key invalid, rate-limited or quota exhausted	Medium	Medium	Provider chain falls back to the next provider; <code>buildError()</code> gives a graceful response; previous session kept
<code>LazyInitializationException</code> regression	Low (fixed)	High	JOIN FETCH in repositories and <code>@Transactional</code> ; TC-08 to TC-11 guard it
Test users (e2e_...@example.com) accumulate in the database	Medium	Low	Run the suite against a separate test database or profile and delete the e2e_ users afterwards
Platform API schema change	Low	Medium	Tolerant parsing; per-platform isolation; fix in one service class
Demo-day network failure	Medium	High	Pre-synchronised demo account and recorded AI session available offline
All free-tier models unavailable at demo time	Medium	High	Pre-generated AI session loaded through <code>/api/ai/sessions/latest</code> ; paid-tier key as backup
Model retired or renamed by a provider	Medium	Low	Run-time model discovery and cooldown; no hard-coded names

10.4 Test Cases and Results

Seventeen automated end-to-end test cases (class `CodePulseE2ETest`) cover authentication, profile, analytics, recent submissions, recommendations, access control, the AI Coach and synchronisation. They were executed on 10 October 2026 against the running application with a real database and a real AI provider.

Table 43: *Test cases*

TC	Description	Input / steps	Expected result	Result
01	Register with valid data	POST /api/auth/register {username, email, password, fullName}	200/201 and a JWT token	PASS
02	Register with duplicate email	Register the same email twice	Second request rejected (4xx)	PASS
03	Login with correct password	POST /api/auth/login	200 and a JWT token	PASS
04	Login with wrong password	POST /api/auth/login with a wrong password	Rejected (4xx)	PASS
05	Protected endpoint without token	GET /api/analytics/me without Authorization	401 or 403	PASS
06	Get own profile	GET /api/users/me with token	200; email equals the registered email	PASS
07	Update profile handles	PUT /api/users/me with codeforcesHandle	2xx	PASS
08	Analytics for a new user	GET /api/analytics/me	200 with totalSubmissions; no HTTP 500	PASS
09	Recent submissions	GET /api/submissions/recent?limit=10	200 and a JSON array	PASS
10	Recommendations list	GET /api/recommendations	200 and a JSON array	PASS
11	Generate recommendations without data	POST /api/recommendations/generate for a user without submissions	2xx; no HTTP 500	PASS
12	Admin endpoint as normal user	GET /api/admin/users with a USER token	403 Forbidden	PASS
13	AI latest session	GET /api/ai/sessions/latest	Not 5xx and not 401/403	PASS
14	AI health endpoint	GET /api/ai/health/groq without and with token	Denied without token; 2xx with token	PASS
15	AI Coach request	POST /api/ai/recommend {prompt, count: 4}	200 with coachInsight and recommendations	PASS
16	AI count cap	POST /api/ai/recommend with count = 50	200 and at most 10 recommendations	PASS
17	Synchronisation starts	PUT handle, then POST /api/analytics/sync	2xx immediately (sync runs asynchronously)	PASS

Results are from the automated run of 10 October 2026 (Maven Surefire: 17 tests run, 0 failures, 0 errors, 0 skipped; BUILD SUCCESS; 116.5 s). The test numbers are those used in CodePulseE2ETest.java.

10.5 Detailed Test Outcomes (Selected Cases)

The console log of the run shows the behaviour behind the verdicts of Table 43. Only the entries relevant to the requirements are quoted.

Table 44: *Execution evidence from the automated run*

Item	Observed evidence (console log of the run)	Verdict
Start-up	Spring Boot 3.3.4 started on Java 17.0.6; Hibernate 6.5.3; 7 JPA repositories found; HikariCP connected to the database; Tomcat started on a random port; OpenRouter free-model discovery found 15 usable models.	Context loaded
TC-11	Log: "No submissions for user ... Skipping recommendation generation." The request completed without an error.	PASS
TC-14	Log: "Groq models available to this key: [qwen/qwen3.8-27b, openai/gpt-oss-120b, allam-2-7b, openai/gpt-oss-20b]"; the best-first order was gpt-oss-120b, gpt-oss-20b, qwen, allam; "Groq success with model openai/gpt-oss-120b".	PASS

Item	Observed evidence (console log of the run)	Verdict
TC-15	Log: "AI provider order for this request: [Groq, Mistral, OpenRouter]"; "Groq succeeded with model openai/gpt-oss-120b in 3183 ms".	PASS
TC-16	Same provider order; Groq answered in 4132 ms; the response held at most 10 recommendations although 50 were requested.	PASS
TC-17	Log: "Syncing Codeforces submissions for handle: tourist"; the HTTP request returned before the sync finished.	PASS
Summary	Tests run: 17, Failures: 0, Errors: 0, Skipped: 0, time 116.5 s, BUILD SUCCESS.	All pass

10.6 Requirements Traceability

Table 45: Requirements-to-test traceability

Requirement	Test cases	Requirement	Test cases
FR-1, FR-2	TC-01, 02, 03, 04	FR-17, FR-18, FR-19	TC-15, 16
FR-3, FR-4	TC-06, 07	FR-20, FR-21	TC-13, 15
FR-5	TC-17 (sync starts)	FR-22	Not covered by the automated suite
FR-9, FR-10	TC-08, 09	FR-23, FR-24, FR-25	TC-14, 15 (provider chain and model discovery in the log)
FR-14, FR-15	TC-10, 11	FR-26	Not covered by the automated suite
FR-6, FR-7, FR-8, FR-11, FR-12, FR-13, FR-16	Not covered by the automated suite	FR-27	TC-14
NFR-2 (security)	TC-05, 12	NFR-9 (AI availability)	TC-15, 16

10.7 Test Summary

Table 46: Test summary

Metric	Value
Test cases defined	17
Passed	17
Failed	0 (0 errors, 0 skipped)
Defects found and fixed during testing	LazyInitializationException on three endpoints; AI JSON parsing failures (fixed by AiJsonUtil) after provider schema change
Areas needing continued attention	Automated tests for pruning, rating bounds, streak and weakness ranking, recommendation solve/dismiss, the AI provider cooldowns and the frontend components; load testing beyond a single user.
Execution	10 October 2026; mvn -Dtest=CodePulseE2ETest test; 17 run, 0 failures, 0 errors, 0 skipped; BUILD SUCCESS (116.5 s)

11. User Manual

This manual explains how an end user installs nothing but a browser: once the system is running (Chapter 12), everything is done through the web interface at <http://localhost:3000>.

11.1 Getting Started

11.1.1 Create an Account

1. Open <http://localhost:3000>. You are redirected to the **Login** page.
2. Click **Create Account**. Step 1 asks for a username and your full name; click **Next**.
3. Step 2 asks for your email and a password; click **Launch**.

- You are signed in automatically and land on the Dashboard.

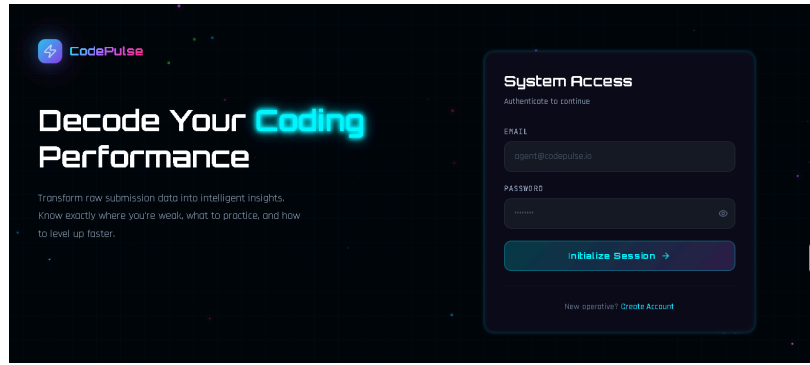


Figure 34: Login and registration screens

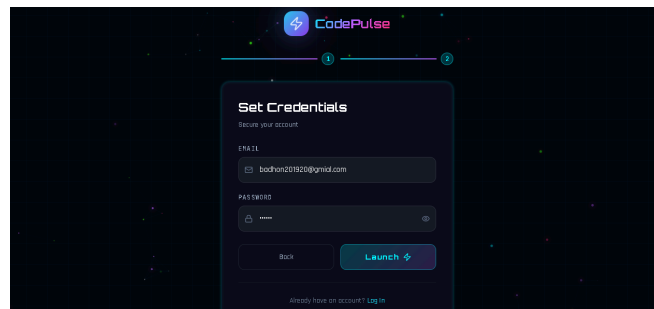


Figure 35: Registration (step 2: credentials)

11.1.2 Sign In

Enter your email and password and press **Sign In**. A wrong password shows an "Invalid credentials" message. Your session lasts 24 hours; after that you are sent back to the login page.

11.1.3 Connect Your Platforms

- Click **Profile** in the sidebar.
- Enter your handle for each platform you use - for example **tourist** for Codeforces. You may connect one, several or all four.
- Click **Save Changes**. Connected platforms show a green status dot, and a link to your public profile.

11.1.4 Synchronise Your Data

- Open the **Dashboard**.
- Click **Sync All Platforms**. A spinner shows while data is fetched in the background.
- After a few seconds the page refreshes with your submissions, rating and streak.

CodePulse also synchronises automatically every 6 hours. Each sync fetches up to 200 recent submissions per platform and the system keeps your latest 300 submissions in total.

11.2 Using the Application

11.2.1 Dashboard

The Dashboard is your overview. The four tiles show your **CodePulse Rating**, **problems solved**, **current streak** and **acceptance rate**. The Platform Connection Grid shows each connected handle with its solved count; click a handle to open your profile on that site, or "Add handle →" to connect a new one. The Recent Submissions list shows your latest attempts with verdict badges; click a title to open the problem.

11.2.2 Analytics

The Analytics page explains **why** your numbers look the way they do.

- **Insights panel:** automatically generated findings, colour-coded as Critical, Warning, Success, Tip or Pattern - for example "You have 0 accepted DP submissions - Blind Spot".
- **Charts:** platform comparison, Codeforces rating trend, difficulty distribution, weakness radar, verdict distribution and a 365-day activity heatmap (hover a cell for date and count).
- **Topic coverage:** all topics you have solved problems in, sorted by volume.

11.2.3 Curated Recommendations

1. Open **Recommendations** and click **Generate New**. Problems are chosen from your weakest topics at a suitable difficulty; older recommendations are replaced.
2. Use the **All / CF / LC / AC / CC** tabs to filter by platform.
3. Click a problem title to open it on the platform.
4. Click **✓ Solved** once you have solved it, or **✗ Dismiss** to hide it. This feedback refines future suggestions.

11.2.4 AI Coach

1. Open **AI Coach** and describe your goal, e.g. "I keep failing DP problems - give me a progression from easy to hard." You can also click an example chip.
2. Choose how many problems you want (4, 6 or 8) and click **Ask AI** (or press Ctrl+Enter).
3. Wait while the animated dots show that the coach is working.
4. Read the **Coach Insight**, the **Focus Areas** and the recommended problems, each with a reason written for you.
5. Solve or dismiss items. Your latest session is shown whenever you return; ask again with a new prompt to replace it.

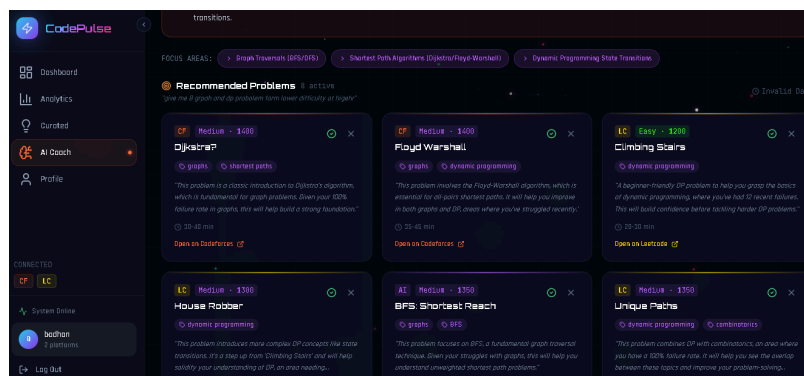


Figure 36: AI Coach

11.2.5 Profile

Edit your full name, avatar URL and any of the four handles, then click **Save Changes**. Changes appear immediately on the Dashboard because every page loads your profile fresh from the server.

11.2.6 Administrator Functions

Users with the ADMIN role can list all users and deactivate accounts through the admin API (Chapter 13). Ordinary users receive 403 Forbidden for these endpoints.

11.3 Understanding the Numbers

Table 47: Glossary of displayed metrics

Term	Meaning
CodePulse Rating	A single Elo-like score (800–3800) that blends your estimated rating on every connected platform, weighted by how much you practise on each.
Acceptance rate	Accepted submissions divided by total submissions in your stored window.
Current / longest streak	Consecutive days with at least one submission on any platform.

Term	Meaning
Weakness score	Failure rate on a topic: 0.0 means strong, 1.0 means you fail every attempt.
Difficulty tiers	Beginner, Easy, Medium, Hard and Expert, mapped from each platform's own scale.

11.4 Troubleshooting

Table 48: Troubleshooting guide

Issue	Resolution
Handles not showing on the Dashboard	Click Save Changes on the Profile page, then return to the Dashboard.
Sync shows 0 new submissions	Check the handle spelling on the Profile page. LeetCode occasionally rate-limits; wait a minute and retry.
AI recommendations are empty	Check that at least one AI provider key (app.groq.api-key, mistral.api-key or app.openrouter.api-key) is set in application.properties. The coach tries every configured provider and model automatically; if all are rate-limited the page shows a friendly message and the previous session is kept, so wait a few minutes and try again.
"Invalid credentials" on login	Check email and password; passwords are case-sensitive.
Redirected to login unexpectedly	Your 24-hour session expired; sign in again.
500 Internal Server Error	Check the backend log; ensure you are running the latest build with the lazy-loading fixes.
Charts are empty	You have no synchronised data yet - connect a handle and click Sync All Platforms.

12. Installation and Deployment Guide

No separate installer is provided. The application is installed from source or by running the packaged executable JAR together with the Next.js application.

12.1 Prerequisites

Table 49: Prerequisites

Software	Version	Purpose
JDK	17 or newer	Run the Spring Boot backend
Maven	3.9+	Build the backend
PostgreSQL	15+	Database
Node.js	18+	Build and run the frontend
Git	Any recent	Clone the repository
AI provider key	At least one	AI Coach (free tiers: console.groq.com, console.mistral.ai, openrouter.ai)

12.2 Obtain the Source

```
git clone https://github.com/Badhon1401/SPL-3.git
cd SPL-3
```

12.3 Backend

1. Create the database:

```
CREATE DATABASE codepulse;
```

2. Generate a JWT secret (Base64, 256-bit):

```
openssl rand -base64 32
```

3. Edit `src/main/resources/application.properties`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/codepulse
spring.datasource.username=postgres
spring.datasource.password=YOUR_PASSWORD
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
jwt.secret=YOUR_BASE64_SECRET
jwt.expiration=86400000
app.groq.api-key=gsk_YOUR_GROQ_KEY
mistral.api-key=YOUR_MISTRAL_KEY
app.openrouter.api-key=sk-or-v1-YOUR_KEY
```

4. Run:

```
mvn spring-boot:run
```

The server starts on <http://localhost:8080>. The 20 algorithm topics are seeded automatically on first boot. To produce a deployable artifact:

```
mvn clean package -DskipTests
java -jar target/codepulse-1.0.0.jar
```

12.4 Frontend

```
cd frontend
npm install
cp .env.local.example .env.local      # set NEXT_PUBLIC_API_URL=http://localhost:8080
npm run dev
```

Open <http://localhost:3000>; you are redirected to the login page. For a production build use `npm run build` followed by `npm start`.

12.5 Verifying the Installation

Table 50: Smoke test

Check	Expected result
Backend log	Application started on port 8080; topics seeded
<code>POST /api/auth/register</code> via Postman	201 with token
Open http://localhost:3000	Login page renders
Register, save a handle, click Sync	Dashboard shows submissions and rating

13. API Reference

All endpoints except `/api/auth/**` require the header `Authorization: Bearer <token>`. Errors use a common `ApiResponse` envelope with an HTTP status code and message.

13.1 Endpoint Summary

Table 51: REST endpoints

Method	Endpoint	Description	Auth
POST	<code>/api/auth/register</code>	Create an account	Public
POST	<code>/api/auth/login</code>	Authenticate, receive JWT	Public
GET	<code>/api/users/me</code>	Get own profile with handles	JWT
PUT	<code>/api/users/me</code>	Update profile and handles	JWT

Method	Endpoint	Description	Auth
GET	/api/analytics/me	Full combined analytics	JWT
POST	/api/analytics/sync	Trigger multi-platform sync (async)	JWT
GET	/api/submissions/recent	Recent submissions, all platforms (<code>?limit=</code>)	JWT
GET	/api/recommendations	Active curated recommendations	JWT
POST	/api/recommendations/generate	Generate new recommendations	JWT
PATCH	/api/recommendations/{id}/solved	Mark solved	JWT
PATCH	/api/recommendations/{id}/dismiss	Dismiss	JWT
POST	/api/ai/recommend	AI Coach: generate from prompt	JWT
GET	/api/ai/sessions/latest	Load stored AI session	JWT
PATCH	/api/ai/items/{id}/solved	Mark AI item solved	JWT
PATCH	/api/ai/items/{id}/dismiss	Dismiss AI item	JWT
GET	/api/admin/users	List all users	JWT + ADMIN
DELETE	/api/admin/users/{id}	Deactivate user	JWT + ADMIN
GET	/api/ai/health/{provider}	Self-test of groq, mistral or openrouter	JWT

14. Conclusion and Deliverables

14.1 Summary of Achievements

CodePulse meets its objectives: it aggregates four competitive-programming platforms into one analytical view, computes a unified rating, identifies topic-level weaknesses, generates insights that users would not otherwise notice, recommends problems algorithmically and - through a grounded LLM pipeline - converts free-text goals into personalised practice plans. The system is secured with JWT and BCrypt, protects its database from unbounded growth, tolerates partial failure of external services and is covered by a 17-case automated end-to-end test suite in which all cases pass.

14.2 Repository and Deliverables

Table 52: *Final deliverables*

Deliverable	Location
Public version-controlled repository	https://github.com/Badhon1401/SPL-3
Backend source and README	Repository root (Maven project)
Frontend source	<code>frontend/</code> directory
Final report	This document
Installer	Not applicable - run from source / executable JAR (Chapter 12)
Demonstration (acceptance test)	Live demo using the scenarios in Chapter 10 and Chapter 11

References

- [1] Spring Boot Reference Documentation, 3.3.x - <https://docs.spring.io/spring-boot/>
- [2] Hibernate ORM 6.5 User Guide - <https://hibernate.org/orm/documentation/6.5/>
- [3] PostgreSQL 15 Documentation - <https://www.postgresql.org/docs/15/>
- [4] Neon Serverless Postgres Documentation - <https://neon.tech/docs>
- [5] Next.js 14 Documentation - <https://nextjs.org/docs>

- [6] Codeforces API - <https://codeforces.com/apiHelp>
- [7] LeetCode GraphQL endpoint (community documentation) - <https://leetcode.com/graphql>
- [8] AtCoder Problems API (kenkoooo) - <https://kenkoooo.com/atcoder/resources>
- [9] Groq API Reference (OpenAI-compatible chat completions, models) - <https://console.groq.com/docs>
- [10] Mistral AI API Documentation - <https://docs.mistral.ai>
- [11] OpenRouter API Reference (models, chat completions) - <https://openrouter.ai/docs>
- [12] SE-801 Final Presentation guidelines, Institute of Information Technology, University of Dhaka, 2026.
- [13] CodePulse source repository - <https://github.com/Badhon1401/SPL-3>